

Operating System

Prepared By:

Er. Ayush Chaudhary

Chapter 4

Memory Management

Contents:

Introduction:.....	5
Memory Hierarchy.....	6
Monoprogramming vs. Multi-programming:.....	8
Swapping	12
Memory Management with Bitmaps.....	14
Memory Management with Linked List	15
Memory Allocation Techniques	17
Contiguous Memory Allocation.....	18
Fixed-size Partition Scheme	19
Advantages of Fixed-size Partition Scheme	21
Disadvantages of Fixed-size Partition Scheme.....	21
Variable-size Partition Scheme	22
Advantages of Variable-size Partition Scheme	23
Disadvantages of Variable-size Partition Scheme	24
Fragmentation:.....	27
Internal fragmentation:	27

External Fragmentation:	28
Reallocation & Protection:.....	32
Memory Allocation Strategies (Dynamic Partitioning Placement Algorithm):	35
Virtual Memory:	41
Paging:	43
Structure of Page Table:	47
Hierarchical Paging:	51
Hashed Page Tables:	54
Inverted Page Tables:.....	55
Shared Page Table:.....	57
Demand Paging:.....	58
Handling Page Fault:	60
Translation Lookaside Buffer (TLB):	63
Page Replacement:.....	65
Page Replacement Algorithm:	67
First In First out Page replacement algorithm:.....	67
FIFO Anomaly:	69
Optimal Page Replacement:.....	71
Least Recently Used (LRU) Algorithm:	72
Most Recently Used (MRU):.....	73
Least Frequently Used (LFU):.....	74

Second-Chance Page Replacement Algorithm:	75
Working Set Page Replacement Algorithm:	75
Concept of Locality of Reference:.....	76
Segmentation:	79
Need of Segmentation:	82
Segmentation with Paging:	84

Introduction:

- Memory management is the functionality of an operating system that handles or manages primary memory.
- Memory management keeps track of each and every memory location either it is allocated to some process or it is free.
- It checks how much memory is to be allocated to processes.
- It decides which process will get memory at what time.
- It tracks whenever some memory gets freed or unallocated and correspondingly it updates the status.
- Address binding of instructions and data to memory addresses can happen at three different stages
 - Compile time --When it is known at compile time where the process will reside, compile time binding is used to generate the absolute code.
 - Load time --When it is not known at compile time where the process will reside in memory, then the compiler generates re-locatable code.
 - Execution time --If the process can be moved during its execution from one memory segment to another, then binding must be delayed to be done at run time

Memory Hierarchy

The Computer memory hierarchy looks like a pyramid structure which is used to describe the differences among memory types. It separates the computer storage based on hierarchy.

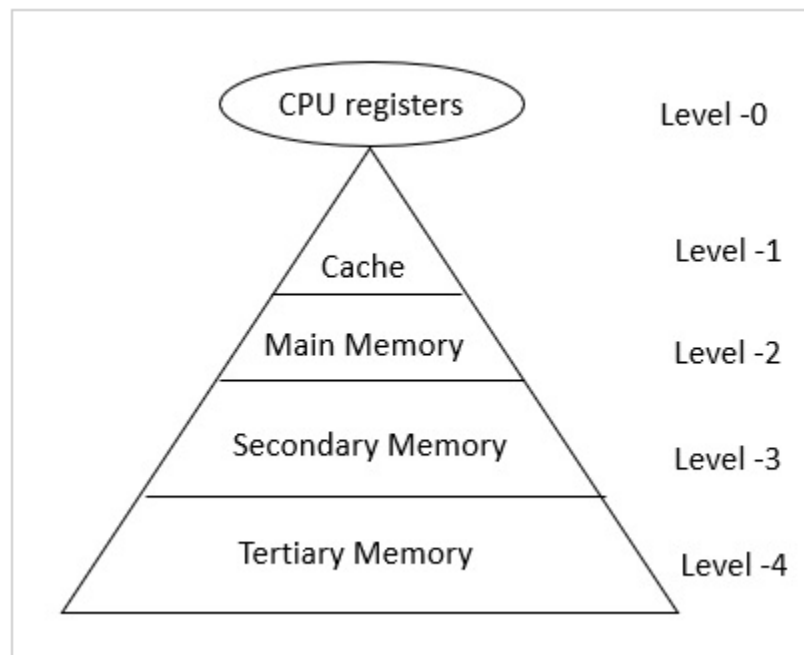
Level 0: CPU registers

Level 1: Cache memory

Level 2: Main memory or primary memory

Level 3: Magnetic disks or secondary memory

Level 4: Optical disks or magnetic tapes or tertiary Memory



In Memory Hierarchy the cost of memory, capacity is inversely proportional to speed. Here the devices are arranged in a manner Fast to slow, that is from register to Tertiary memory.

Let us discuss each level in detail:

Level-0 – Registers

The registers are present inside the CPU. As they are present inside the CPU, they have least access time. Registers are most expensive and smallest in size generally in kilobytes. They are implemented by using Flip-Flops.

Level-1 – Cache

Cache memory is used to store the segments of a program that are frequently accessed by the processor. It is expensive and smaller in size generally in Megabytes and is implemented by using static RAM.

Level-2 – Primary or Main Memory

It directly communicates with the CPU and with auxiliary memory devices through an I/O processor. Main memory is less expensive than cache memory and larger in size generally in Gigabytes. This memory is implemented by using dynamic RAM.

Level-3 – Secondary storage

Secondary storage devices like Magnetic Disk are present at level 3. They are used as backup storage. They are cheaper than main memory and larger in size generally in a few TB.

Level-4 – Tertiary storage

Tertiary storage devices like magnetic tape are present at level 4. They are used to store removable files and are the cheapest and largest in size (1-20 TB).

Monoprogramming vs. Multi-programming:

1. Monoprogramming:

- **Definition:** Also known as single-programming, monoprogramming allows only one program to run at a time.
- **Execution Model:** The entire system resources, including CPU time and memory, are dedicated to a single program until it completes or encounters an interruption.
- **Resource Utilization:** Monoprogramming tends to have lower resource utilization because the CPU and memory are exclusively reserved for the currently running program.

- **Example:** Traditional batch processing systems where one job is executed at a time without interruption.

2. Multi-programming:

- **Definition:** Multi-programming allows multiple programs to run concurrently, sharing the system resources.
- **Execution Model:** Several programs are loaded into memory simultaneously, and the operating system manages the execution of these programs by quickly switching between them.
- **Resource Utilization:** Multi-programming aims to improve resource utilization by keeping the CPU busy with some task at all times. While one program is waiting for I/O, the CPU can switch to executing another program.
- **Example:** Modern operating systems that support concurrent execution of multiple applications or processes.

3. Concurrency and Parallelism:

- **Monoprogramming:** There is no concurrency in monoprogramming. Only one program runs at a time.

- **Multi-programming:** Multiple programs can execute concurrently, but it doesn't necessarily imply true parallelism. The CPU time is shared among programs through time-sharing or interleaved execution.

4. Efficiency:

- **Monoprogramming:** Simple to manage, but resource utilization may not be optimal, especially when a program has to wait for I/O operations.
- **Multi-programming:** More complex management is required to handle multiple programs, but it can lead to better overall system efficiency by keeping the CPU busy during I/O wait times.

Monoprogramming

Pros:

- Simpler to design and implement.
- No need for complex memory management or scheduling algorithms.

- Cons:
 - Poor CPU utilization. The CPU sits idle whenever the program is waiting for I/O operations.
 - Not suitable for interactive use as users have to wait for the current program to finish before they can run another program.
 - Inefficient use of system resources.

Multi-programming

- Pros:
 - Improved CPU utilization. The CPU is kept busy even when programs are waiting for I/O operations.
 - Faster response times for interactive users. Users can switch between programs and work on multiple tasks simultaneously.
 - More efficient use of system resources.
- Cons:
 - More complex to design and implement than monoprogramming systems.
 - Programs may experience unpredictable behavior due to competition for resources.

Swapping

If there is **not enough main memory to hold all the currently active processes**, the excess processes must be kept on the disk and brought in to run dynamically.

A process can **be swapped temporarily out of memory to a backing store**, and then brought back into memory for continued execution.

Backing store –fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images.

Roll out, roll in –swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed.

Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped. Let us assume that the user process is of size 100KB and the backing store is a standard hard disk with transfer rate of 1 MB per second. The actual transfer of the 100K process to or from memory will take

$$100\text{KB} / 1000\text{KB per second} = 1/10 \text{ second} = 100 \text{ milliseconds}$$

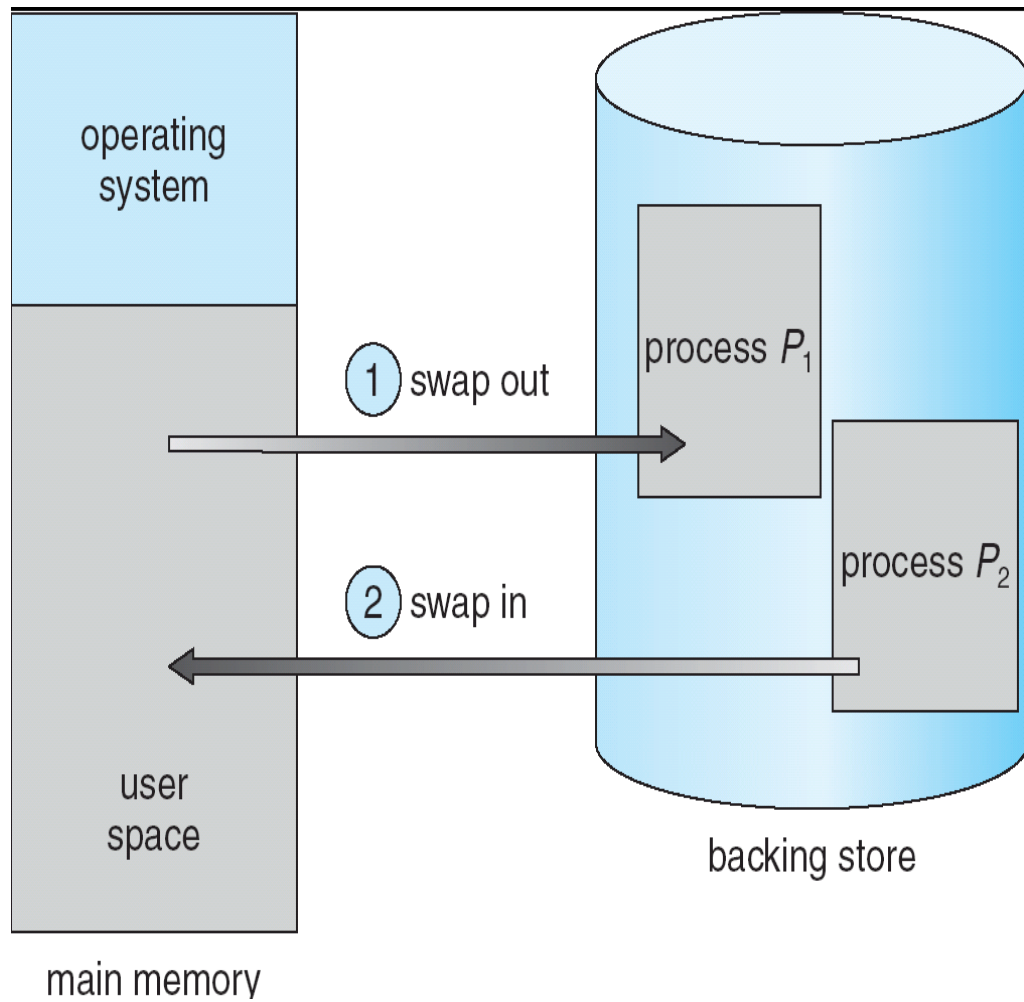


Figure: Swapping of two processes using a disk as a blocking store

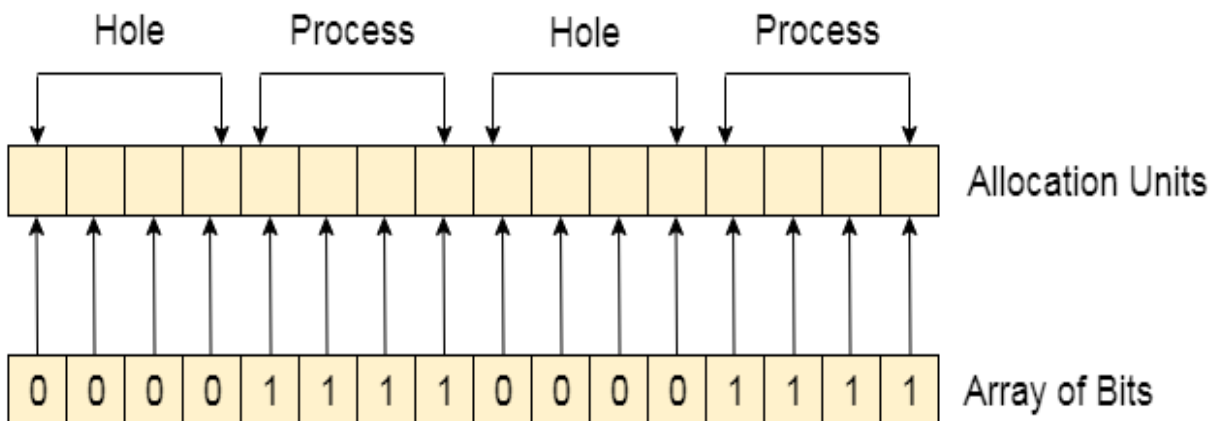
Memory Management with Bitmaps

In bitmap memory management, a bitmap is used to keep track of which blocks of memory are currently in use and which are free.

Each block of memory is represented by a bit in the bitmap, with a value of 1 indicating that the block is currently in use and a value of 0 indicating that the block is free.

Bitmap memory management is efficient in terms of memory usage, as the bitmap itself requires very little memory.

Disadvantage: However, searching for a free block of memory can be time-consuming, as the system needs to scan through the bitmap to find a free block.



Memory Management with Linked List

- Another way to keep track of memory is to maintain a linked list of allocated and free memory segments, where segment either contains a process or is an empty hole between two processes.
- The memory of Fig1(a) is represented in Fig1(c) as a linked list of segments.
- Each entry in the list specifies a hole (H) or process (P), the address at which it starts the length and a pointer to the next entry.

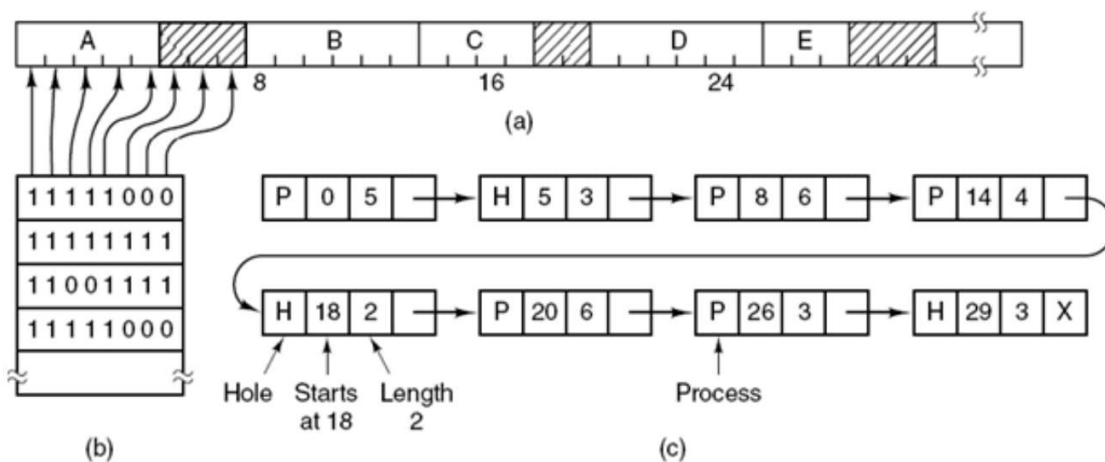


Figure1(a) A part of memory with five processes and three holes. The tick marks show the memory allocation units. The shaded regions (0 in the bitmap) are free. (b) The corresponding bitmap. (c) The same information as a list.

- The segment list is kept sorted by address. Sorting this way has the advantage that when a process terminates or is swapped out, updating the list is straightforward.
- A terminating process normally has two neighbors (except when it is at the very top or bottom of memory).
- These may be either processes or holes, leading to the four combinations of Fig2. In Fig2(a) updating the list requires replacing a P by an H. In Fig2(b) and Fig2(c) two entries are merged into one, and the list becomes one entry shorter.
- In Fig2(d), three entries are merged and two items are removed from the list.

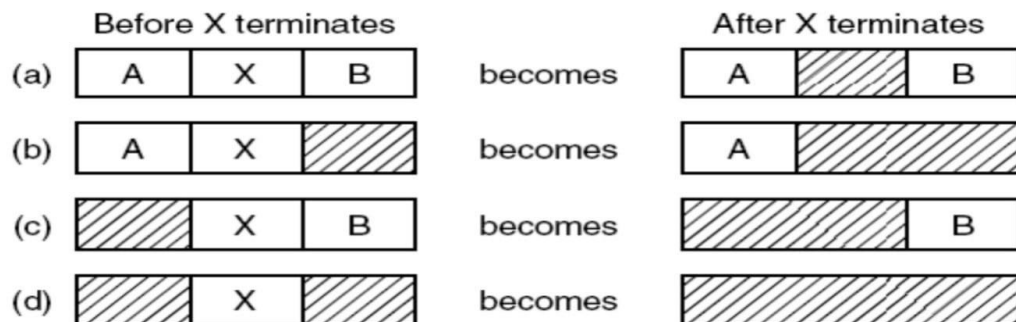


Figure2. Four neighbor combinations for the terminating process X.

Memory Allocation Techniques

- Two types:
 - Contiguous Storage Allocation
 - Fixed Partition Allocation
 - Variable Partition Allocation
 - Non-Contiguous
 - Paging
 - Segmentation

Contiguous Memory Allocation

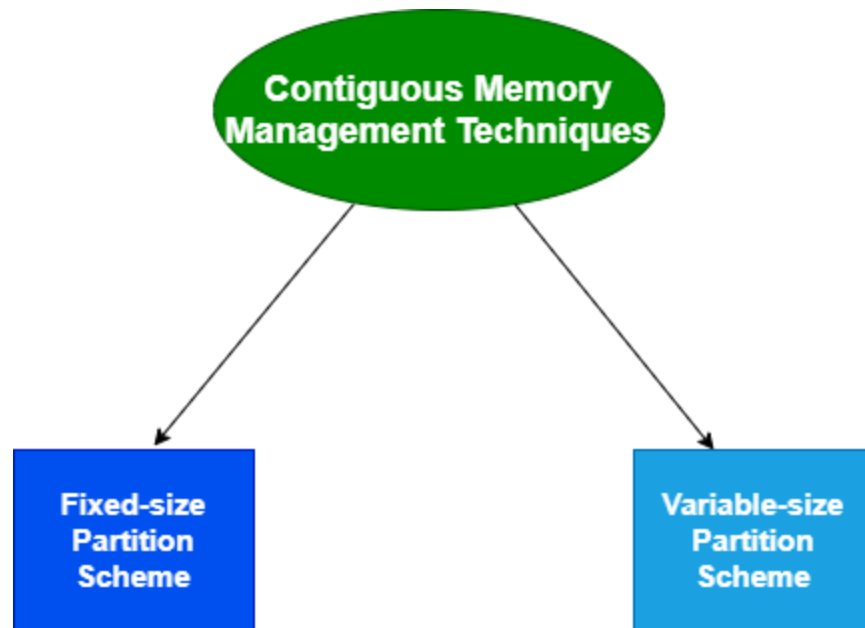
In the Contiguous Memory Allocation, each process is contained in a single contiguous section of memory.

In this memory allocation, all the available memory space remains together in one place which implies that the freely available memory partitions are not spread over here and there across the whole memory space.

In **Contiguous memory allocation** which is a memory management technique, whenever there is a request by the user process for the memory then a single section of the contiguous memory block is given to that process according to its requirement.

Contiguous Memory allocation is achieved just by dividing the memory into the **fixed-sized partition**.

The memory can be divided either in the **fixed-sized partition or in the variable-sized partition** in order to allocate contiguous space to user processes.



Fixed-size Partition Scheme

This technique is also known as **Static partitioning**.

In this scheme, the system divides the memory into fixed-size partitions.

The partitions may or may not be the same size.

The size of each partition is fixed as indicated by the name of the technique and it cannot be changed.

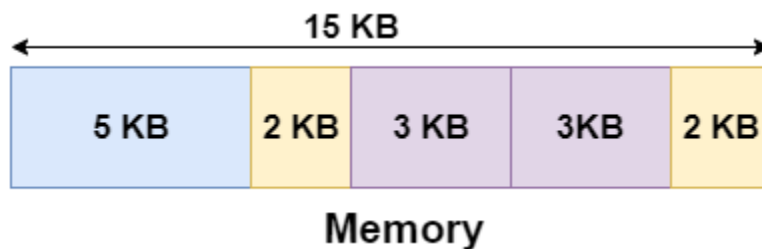
In this partition scheme, each partition may contain exactly one process.

There is a problem that this technique will limit the degree of multiprogramming because the number of partitions will basically decide the number of processes.

Whenever any process terminates then the partition becomes available for another process.

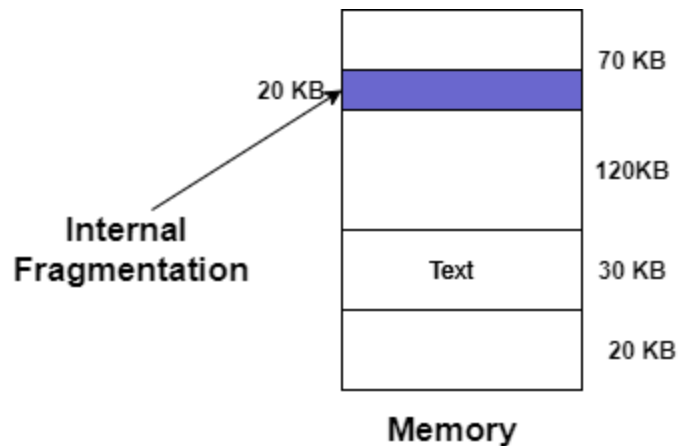
Example:

Let's take an example of fixed size partitioning scheme, we will divide a memory size of 15 KB into fixed-size partitions:



It is important to note that these partitions are allocated to the processes as they arrive and the partition that is allocated to the arrived process basically depends on the algorithm followed.

If there is some wastage inside the partition then it is termed **Internal Fragmentation**.



Advantages of Fixed-size Partition Scheme

- This scheme is simple and is easy to implement
- It supports multiprogramming as multiple processes can be stored inside the main memory.
- Management is easy using this scheme

Disadvantages of Fixed-size Partition Scheme

Some disadvantages of using this scheme are as follows:

1. Internal Fragmentation

Suppose the size of the process is lesser than the size of the partition in that case some size of the partition gets wasted and remains unused. This wastage inside the memory is generally termed as Internal fragmentation

As we have shown in the above diagram the 70 KB partition is used to load a process of 50 KB so the remaining 20 KB got wasted.

2. Limitation on the size of the process

If in a case size of a process is more than that of a maximum-sized partition then that process cannot be loaded into the memory. Due to this, a condition is imposed on the size of the process and it is: the size of the process cannot be larger than the size of the largest partition.

3. External Fragmentation

It is another drawback of the fixed-size partition scheme as total unused space by various partitions cannot be used in order to load the processes even though there is the availability of space but it is not in the contiguous fashion.

4. Degree of multiprogramming is less

In this partition scheme, as the size of the partition cannot change according to the size of the process. Thus the degree of multiprogramming is very less and is fixed.

Variable-size Partition Scheme

This scheme is also known as **Dynamic partitioning** and is came into existence to overcome the drawback i.e internal fragmentation that is caused by **Static partitioning**.

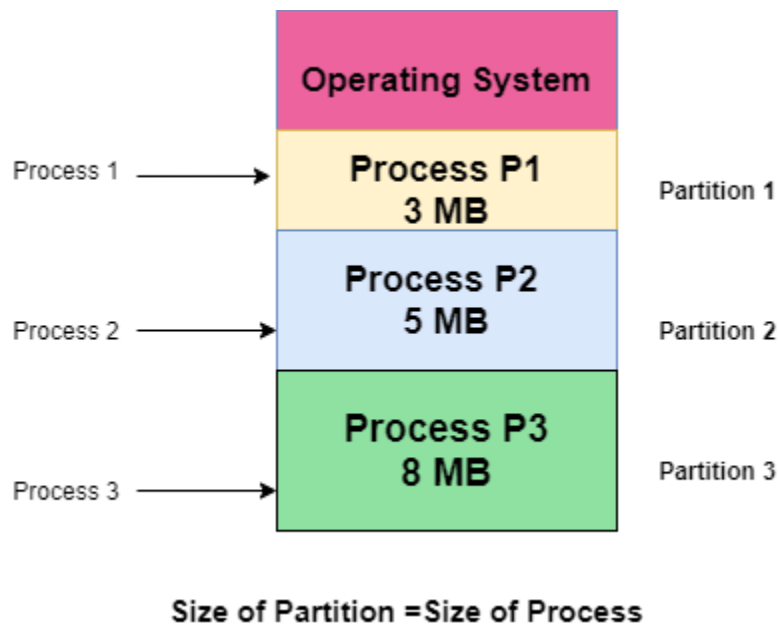
In this partitioning, scheme allocation is done dynamically.

The size of the partition is not declared initially.

Whenever any process arrives, a partition of size equal to the size of the process is created and then allocated to the process.

Thus the size of each partition is equal to the size of the process.

As partition size varies according to the need of the process so in this partition scheme there is no **internal fragmentation**.



Advantages of Variable-size Partition Scheme

Some Advantages of using this partition scheme are as follows:

1. **No Internal Fragmentation** As in this partition scheme space in the main memory is allocated strictly according to the requirement of the process thus there is no chance of internal fragmentation. Also, there will be no unused space left in the partition.
2. **Degree of Multiprogramming is Dynamic** As there is no internal fragmentation in this partition scheme due to which there is no unused space in the memory. Thus more processes can be loaded into the memory at the same time.
3. **No Limitation on the Size of Process** In this partition scheme as the partition is allocated to the process dynamically thus the size of the process cannot be restricted because the partition size is decided according to the process size.

Disadvantages of Variable-size Partition Scheme

Some Disadvantages of using this partition scheme are as follows:

1. **External Fragmentation** As there is no internal fragmentation which is an advantage of using this partition scheme does not mean there will no external fragmentation.

Let us understand this with the help of an example: In the above diagram- process P1(3MB) and process P3(8MB) completed their execution. Hence there are two spaces left i.e. 3MB and 8MB. Let's there is a Process P4 of size 11 MB comes. But the empty space in memory cannot be allocated as no spanning is allowed in contiguous allocation. Because the rule says that process must be continuously present in the main memory in order to get executed. Thus it results in External Fragmentation.

2. Difficult Implementation The implementation of this partition scheme is difficult as compared to the Fixed Partitioning scheme as it involves the allocation of memory at run-time rather than during the system configuration.

As we know that OS keeps the track of all the partitions but here allocation and deallocation are done very frequently and partition size will be changed at each time so it will be difficult for the operating system to manage everything.

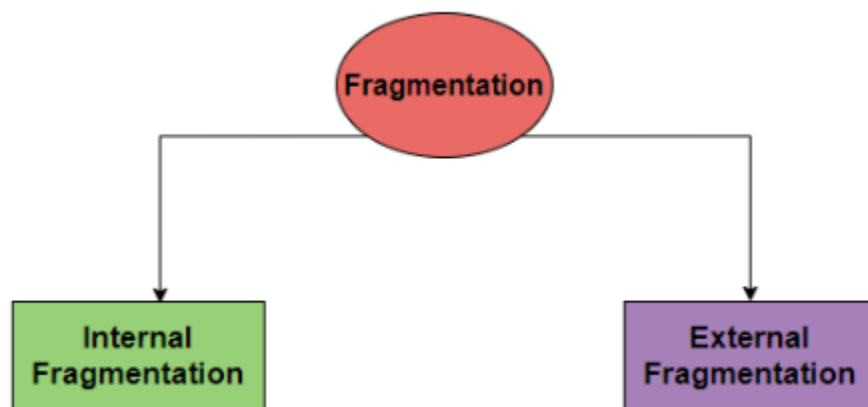
Fixed vs. Variable Partitioning

Fixed Partition	Variable partition
• It is the OS that decides the partition size only once at the system boot time.	• OS has to decide about partition size, every time a new process is chosen by long-term scheduler.
• Here, the degree of multiprogramming is fixed.	• Here, the degree of multiprogramming will vary depending on program size.
• It leads to internal fragmentation.	• It leads to external fragmentation.
• IBM-360 DOS and OS/MFT OS used this approach.	• IBM OS/MFT used this approach also.

Fragmentation:

Fragmentation is a phenomenon that occurs in computer systems when there is **unused memory space between allocated blocks of memory**.

Fragmentation can be divided into two types: **internal fragmentation and external fragmentation**.



Internal fragmentation:

Internal fragmentation occurs when a process requests a block of memory that **is larger than what it actually needs**.

In this case, the operating system allocates a larger block of memory to the process to accommodate its request, but the excess memory is left unused. This leads to **internal fragmentation**.



Figure: Internal Fragmentation

External Fragmentation:

External fragmentation occurs when there is **enough free memory available to satisfy a request, but the memory is not contiguous.**

This happens when the memory space is divided into many small blocks, and none of them are **large enough** to fulfill a request.

As a result, the operating system cannot allocate the requested block of memory, leading to **external fragmentation**.

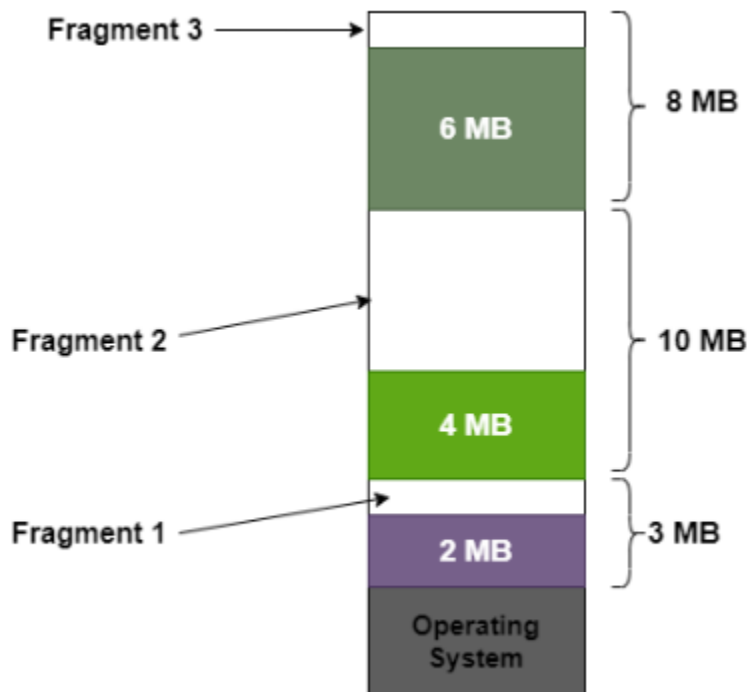


Figure: External Fragmentation

Suppose, we want to allocate memory to a process of **size 8 MB** and as In the above diagram, we can see that, there is **enough space that is 9 MB to run a process of size 8MB** but the memory that is fragments are **not contiguous**.

Coalescing & Compaction:

Coalescing and Compaction are two memory management techniques used to improve memory utilization and **combat external fragmentation**.

Coalescing is the process of merging adjacent blocks of free memory into a larger block.

When memory is allocated and freed frequently, it can result in small, fragmented free memory blocks.

Coalescing helps to reduce fragmentation by merging these small free blocks into larger ones, which can then be more easily utilized for larger memory allocations.

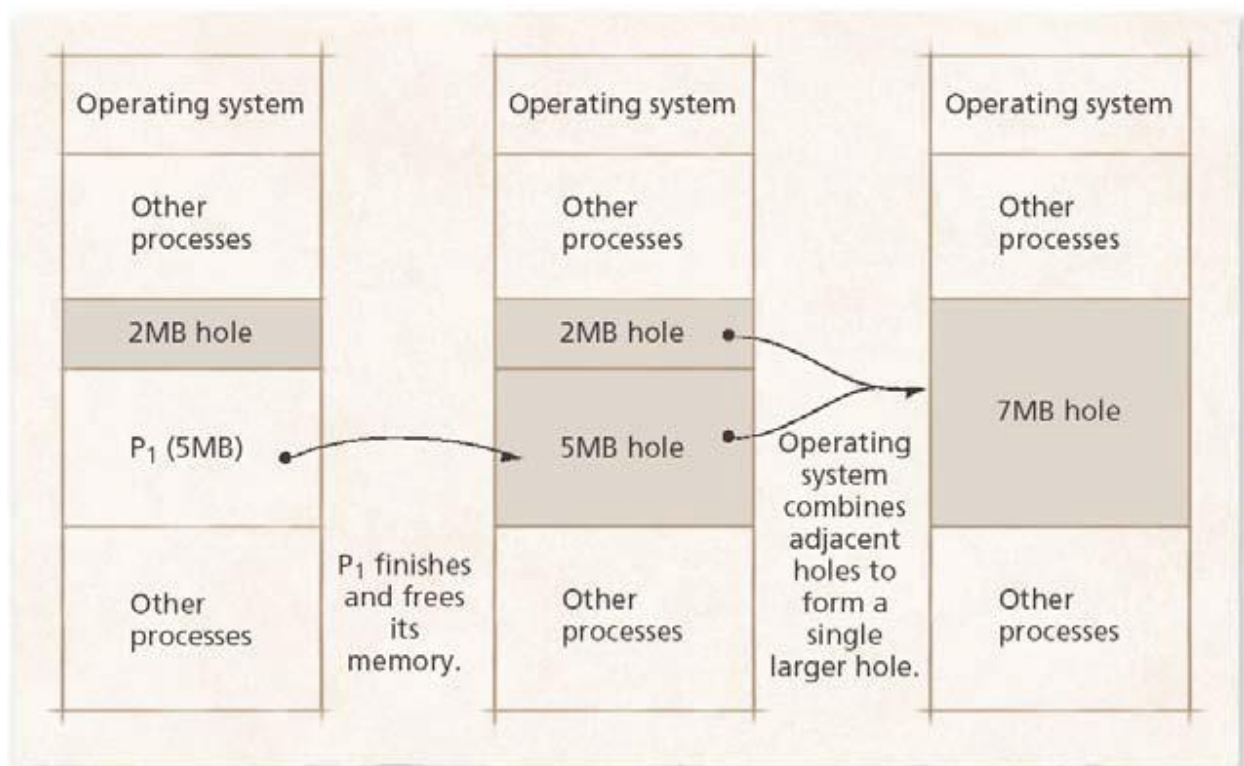


Figure: Coalescing memory “holes”

Compaction:

Compaction is the process of Rearranging memory into a single contiguous block free space and a single contiguous block of occupied space.

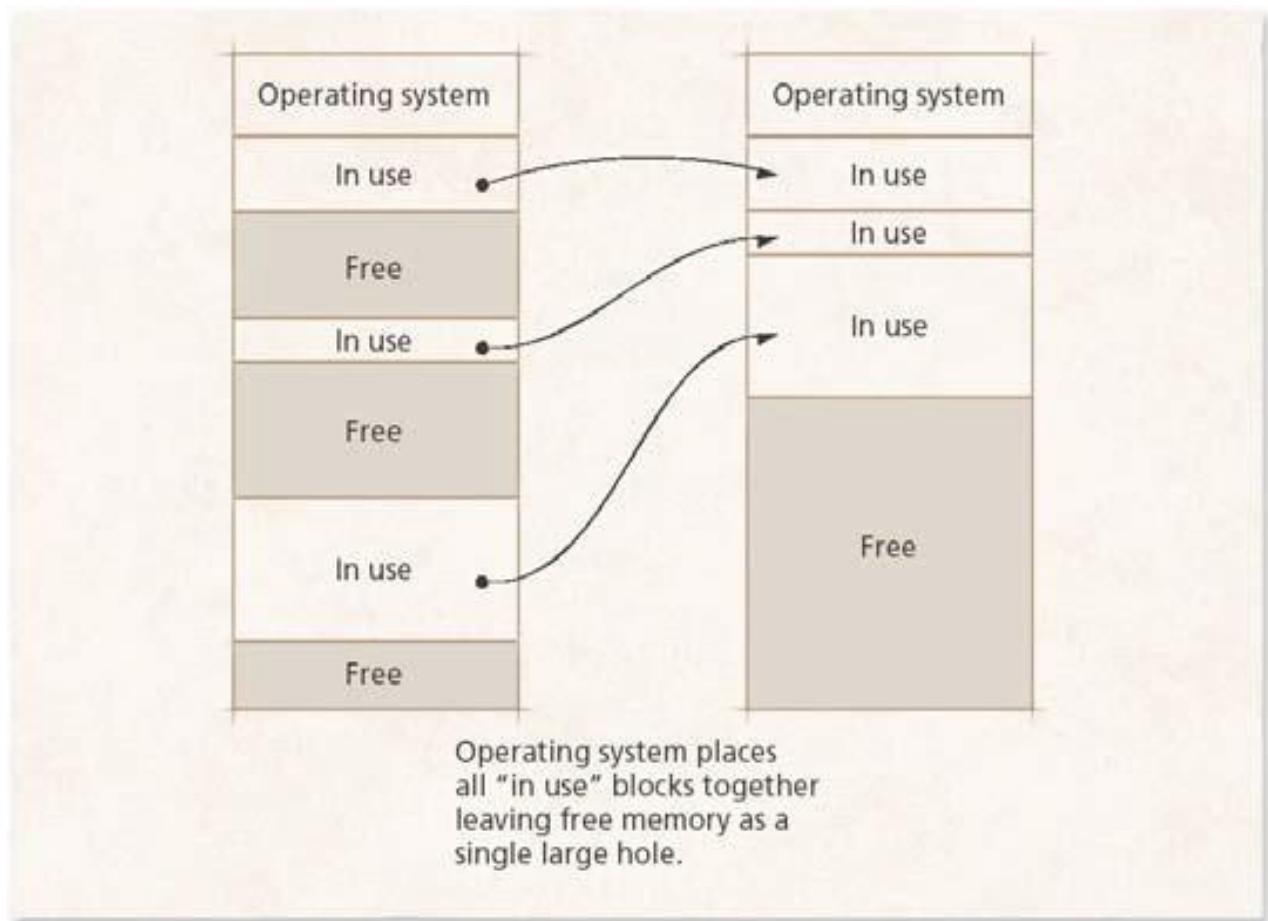


Figure: Memory compaction

Reallocation & Protection:

Program Reallocation:

- Refers to the ability to load and execute a given program into an arbitrary place in the memory.
- Relocation is a mechanism to convert the logical address into a physical address.
- To do this, there is a special register in CPU called relocation register.
- Every address used in the program is relocated as:
$$\text{effective physical address} = \text{Logical address} + \text{Contents of Relocation register}$$
- MMU is a special hardware which performs address binding, uses relocation scheme.

Memory Management Unit (MMU)

- MMU generates physical address from virtual address provided by the program.
- MMU maps virtual addresses to physical addresses and puts them on memory bus

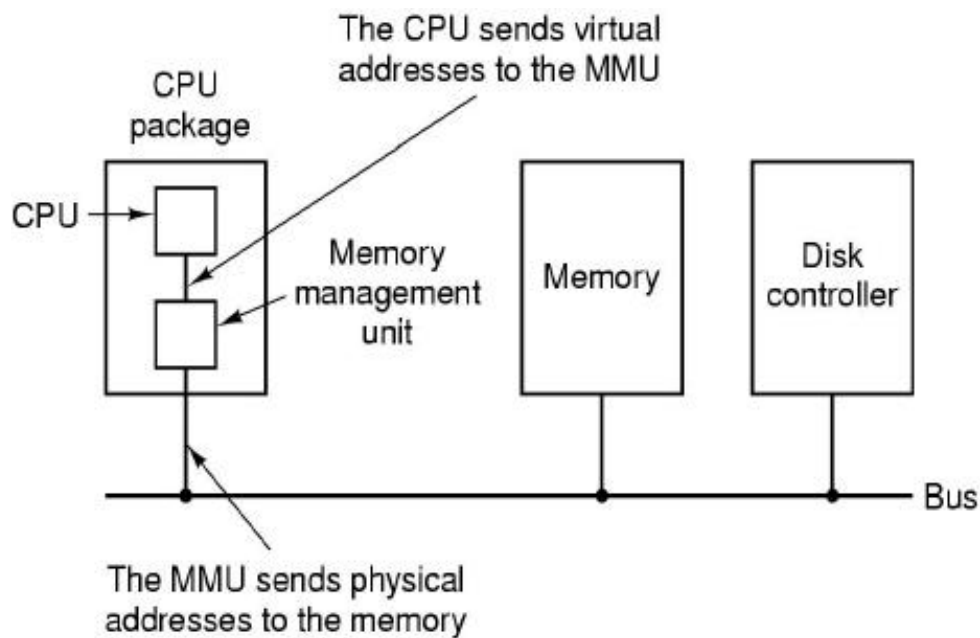
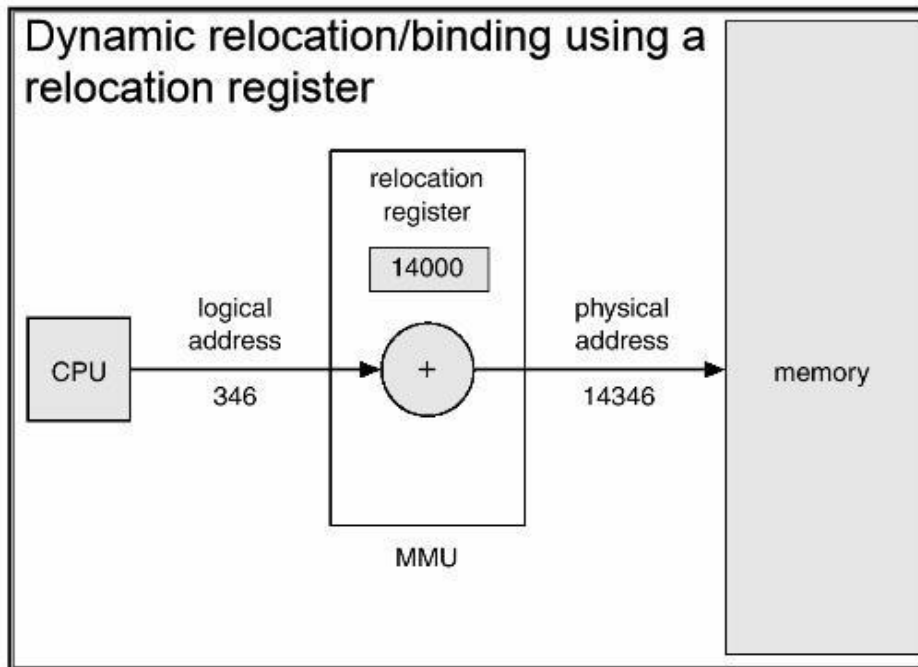


Figure 62: Memory management Unit

- Two basic types of relocation:
 - **Static Relocation:** Formed during the loading of the program into memory by a loader.
 - **Dynamic Relocation:** mapping from the virtual address space to physical address is performed at execution- time.



Protection:

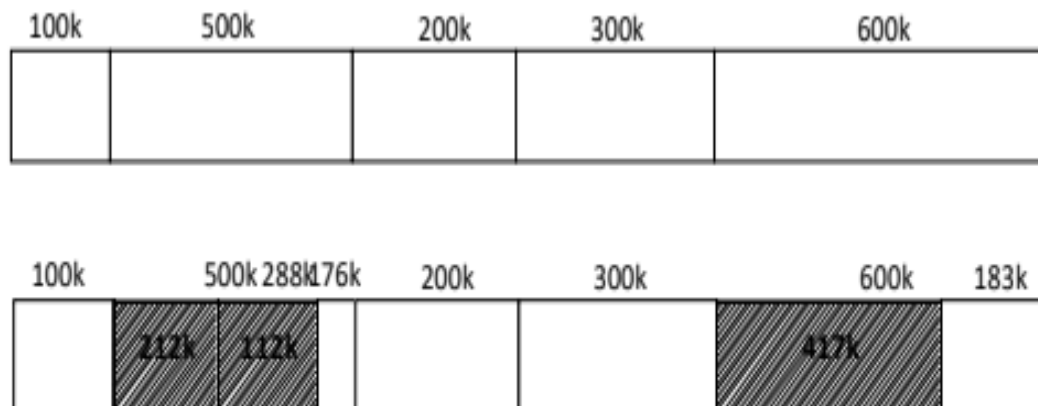
Providing security from unauthorized usage of memory.

- OS can protect the memory with the help of **base and limit registers**.
- **Base register** consist of the starting address of the next process, the **limit register** specifies the boundary of that job.
- That is why the limit register is also called a **fencing register**.

Memory Allocation Strategies (Dynamic Partitioning Placement Algorithm):

First fit:

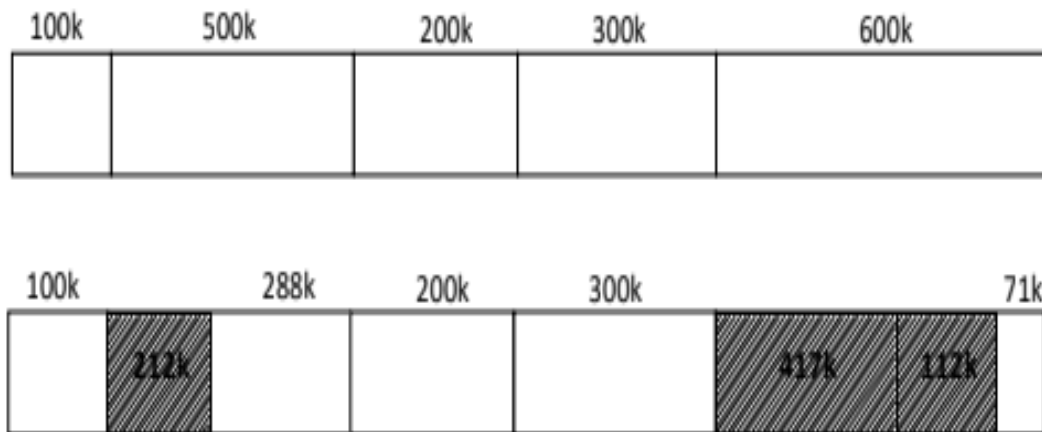
- The memory manager allocates the first hole that is big enough. It stops the searching as soon as it finds a free hole that is large enough.
- Advantages: It is a fast algorithm because it searches as little as possible.
- Disadvantages: Not good in terms of storage utilization.
- Example: Processes of 212K, 417K, 112K, and 426K arrives in order.



- Here Process of size 426K will not get any partition for allocation.

Next fit:

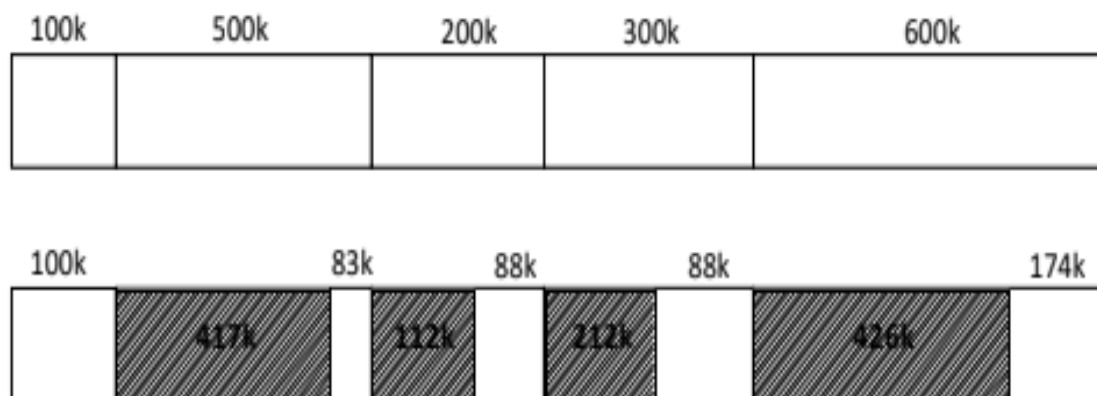
- It works the same way as first fit, except that it keeps track of where it is whenever it finds a suitable hole. The next time it is called to find a hole, it starts searching the list from the place where it left off last time, instead of always at the beginning, as first fit does.
- Example: Processes of 212K, 417K, 112K, and 426K arrives in order.



- Here Process of size 426K will not get any partition for allocation.

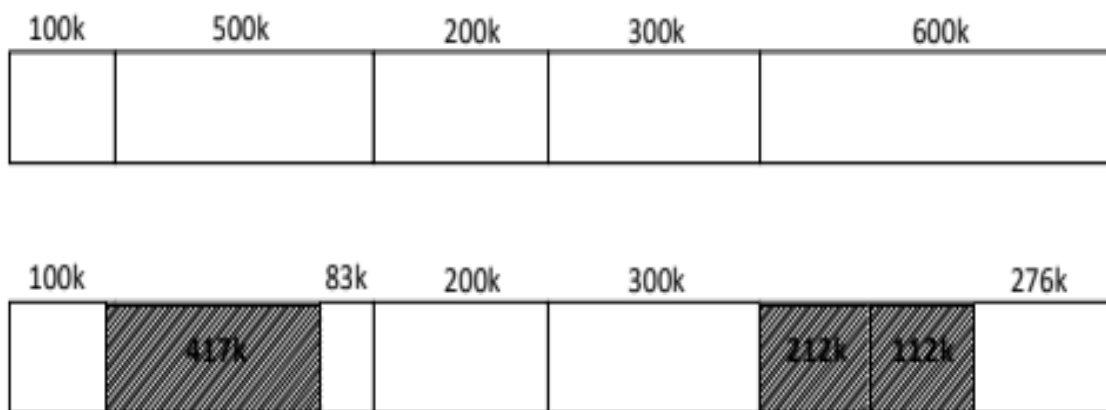
Best fit:

- Allocate the smallest hole that is big enough.
- Best fit searches the entire list and takes the smallest hole that is big enough to hold the new process.
- Best fit try to find a hole that is close to the actual size needed.
- Advantages: more storage utilization than first fit.
- Disadvantages: slower than first fit because it requires searching whole list at time.
- Example: Processes of 212K, 417K, 112K, and 426K arrives in order.



Worst fit:

- Allocate the largest hole.
- It search the entire list, and takes the largest hole, rather than creating a tinny hole, it produces the largest leftover hole, which may be more useful.
- Advantages: some time it has more storage utilization than first fit and best fit.
- Disadvantages: not good for both performance and utilization.
- Example: Processes of 212K, 417K, 112K, and 426K arrives in order.



- Here Process of size 426K will not get any partition for allocation.

Q: Given the memory partitions of 100K, 500K, 200K, 300K and 600K (in order), how would each of the First-fit, Best-fit, and Worst-fit algorithms place processes of 212K, 417K, 112K, and 426K (in order)?

Which algorithm makes the most efficient use of memory?

212
417
112
426

processes

100
500
200
300
600

✦ **First-fit:** search the list of available memory and allocate the first block that is big enough

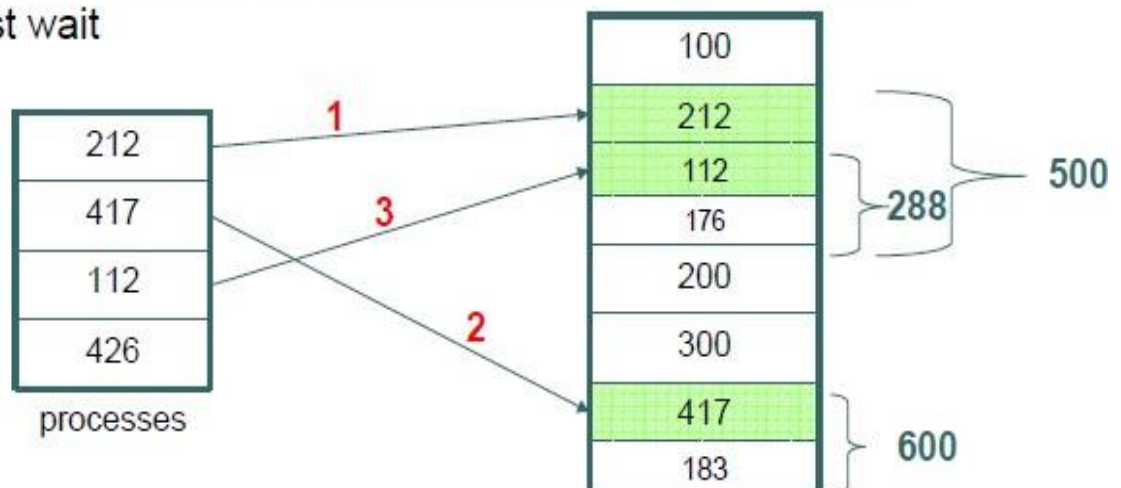
Processes placement:

212 K → 500 K partition

417 K → 600 K partition

112 K → 288 K partition (New partition 288 K = 500 K - 212 K)

426 K must wait



- ✦ **Best-fit:** search the entire list of available memory and allocate the smallest block that is big enough

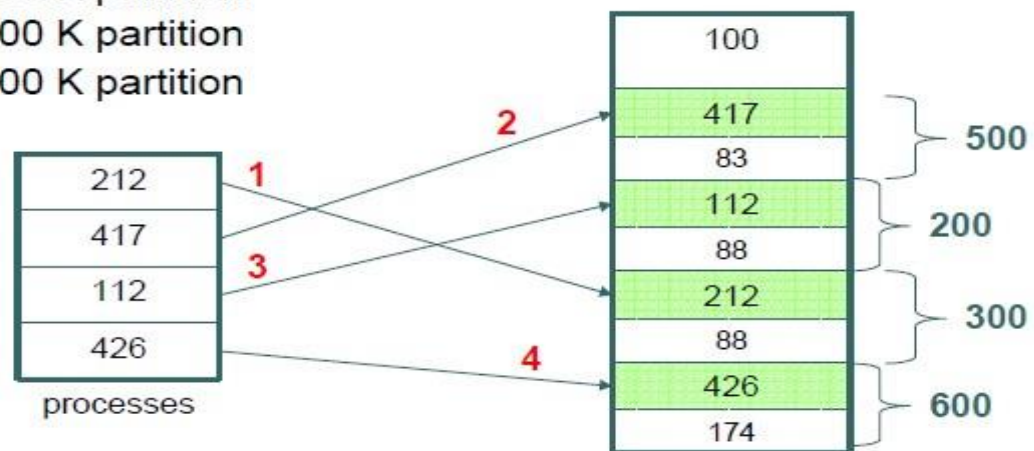
Processes placement:

212 K → 300 K partition

417 K → 500 K partition

112 K → 200 K partition

426 K → 600 K partition



- ✦ **Worst-fit:** search the entire list of available memory and allocate the largest block. The justification for this scheme is that the leftover block produced would be larger and potentially more useful than that produced by the best-fit approach

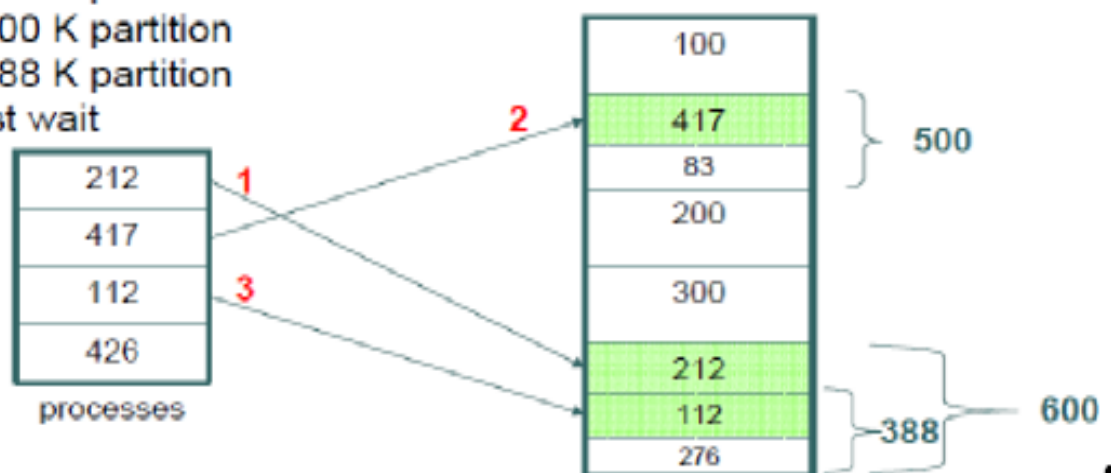
Processes placement:

212 K → 600 K partition

417 K → 500 K partition

112 K → 388 K partition

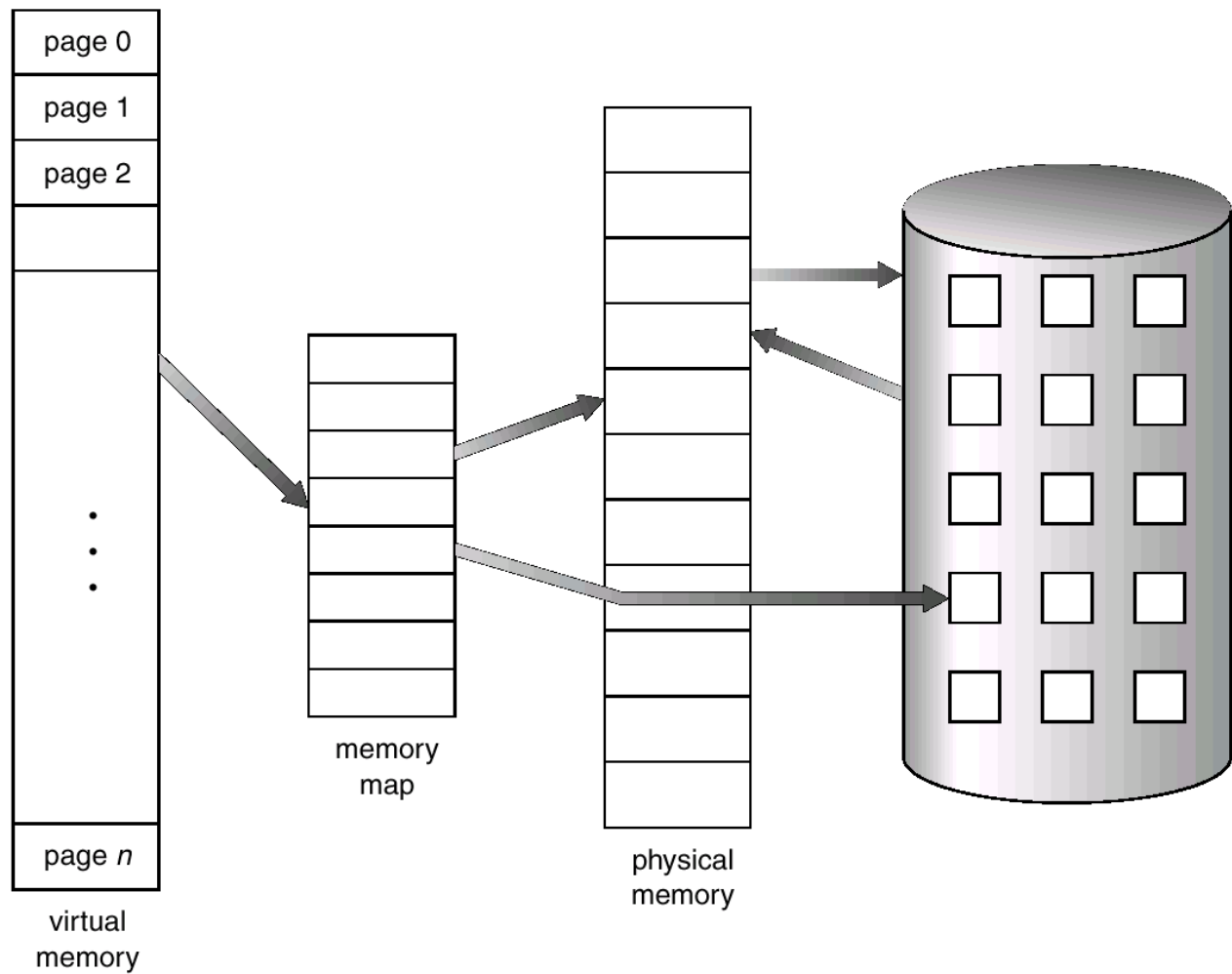
426 K must wait



Virtual Memory:

- Virtual memory is a technique that allows the execution of process that may not be completely in memory. The main visible advantage of this scheme is that programs can be larger than physical memory.
- Solves problem of limited memory space
- Creates the illusion that more memory exists than is available in system
- Two types of addresses in virtual memory systems
 - Virtual addresses
 - Referenced by processes
 - Physical addresses
 - Describes locations in main memory
- Memory management unit (MMU)
 - Translates virtual addresses to physical address

Virtual memory that is larger than the physical memory:



Paging:

- Paging is one of the memory management schemes by which a computer stores and retrieves data from the secondary storage for use in main memory.
- In the paging memory-management scheme, the operating system retrieves data from secondary storage in same-size blocks called pages.
- Paging allows the physical address space of the process to be non contiguous
- Divide physical memory into fixed-sized blocks called frames (size is power of 2, between 512 bytes and 8192 bytes).
- Divide logical memory into blocks of same size called pages.
- Keep track of all free frames.
- To run a program of size n pages, need to find n free frames and load program.
- Set up a page table to translate logical to physical addresses.

Address generated by CPU is divided into:

1. Page number (p) – used as an index into a page table which contains base address of each page in physical memory.
2. Page offset (d) – combined with base address to define the physical memory address that is sent to the memory unit.

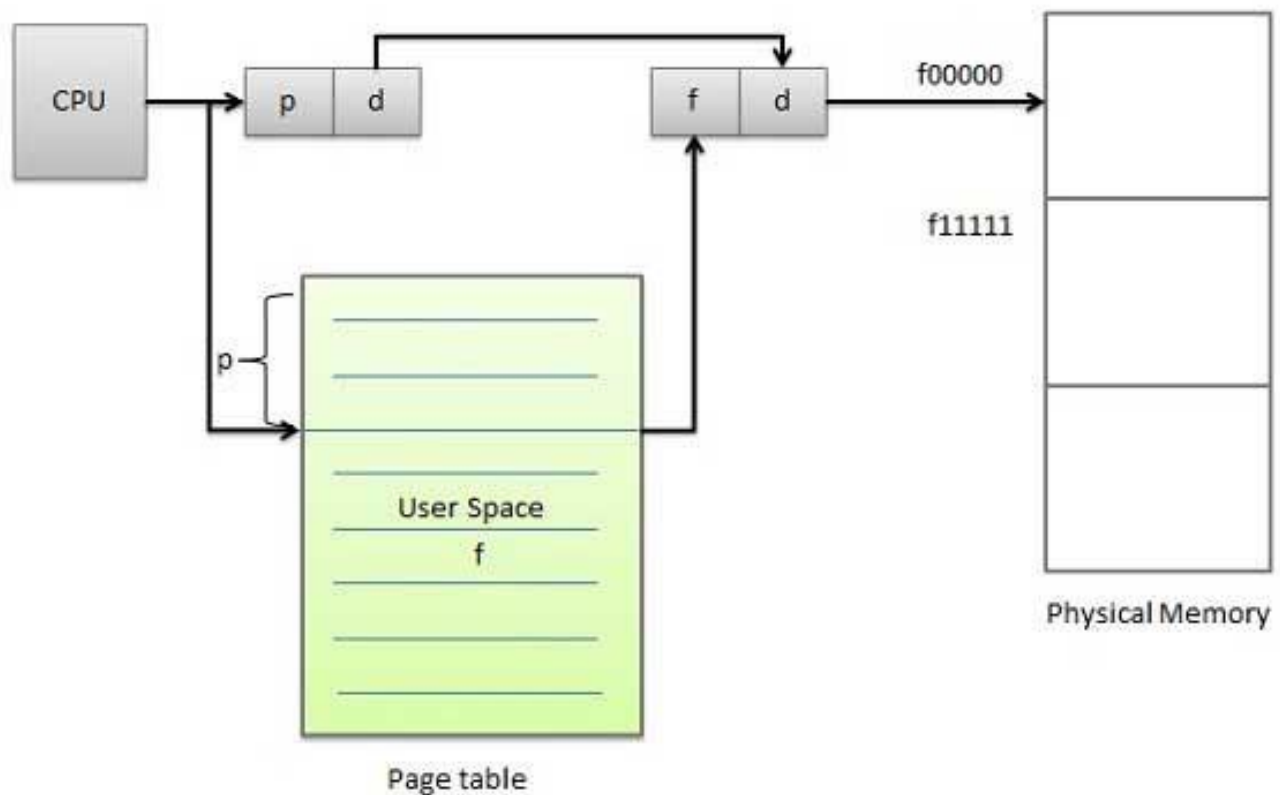


Fig. Address Translation Scheme

- When we use a paging scheme, we have no external fragmentation
- When a process arrives in the system to be executed, its size, expressed in pages, is examined.
- Each page of the process needs one frame. Thus, if the process requires n pages, there must be at least n frames available in memory.
- If there are n frames available, they are allocated to this arriving process.
- The first page of the process is loaded into one of the allocated frames and the frame number is put in the page table for this process.
- The next page is loaded into another frame, and its frame number is put into the page table, and so on.
- The operating system is managing physical memory, it must be aware of the allocation details of physical memory: which frames are allocated, which frames are available, how many total frames there are, and so on. This information is generally kept in a data structure called a frame table.
- The frame table has one entry for each physical page frame, indicating whether the latter is free allocated and, if it is allocated, to which page of which process or processes.

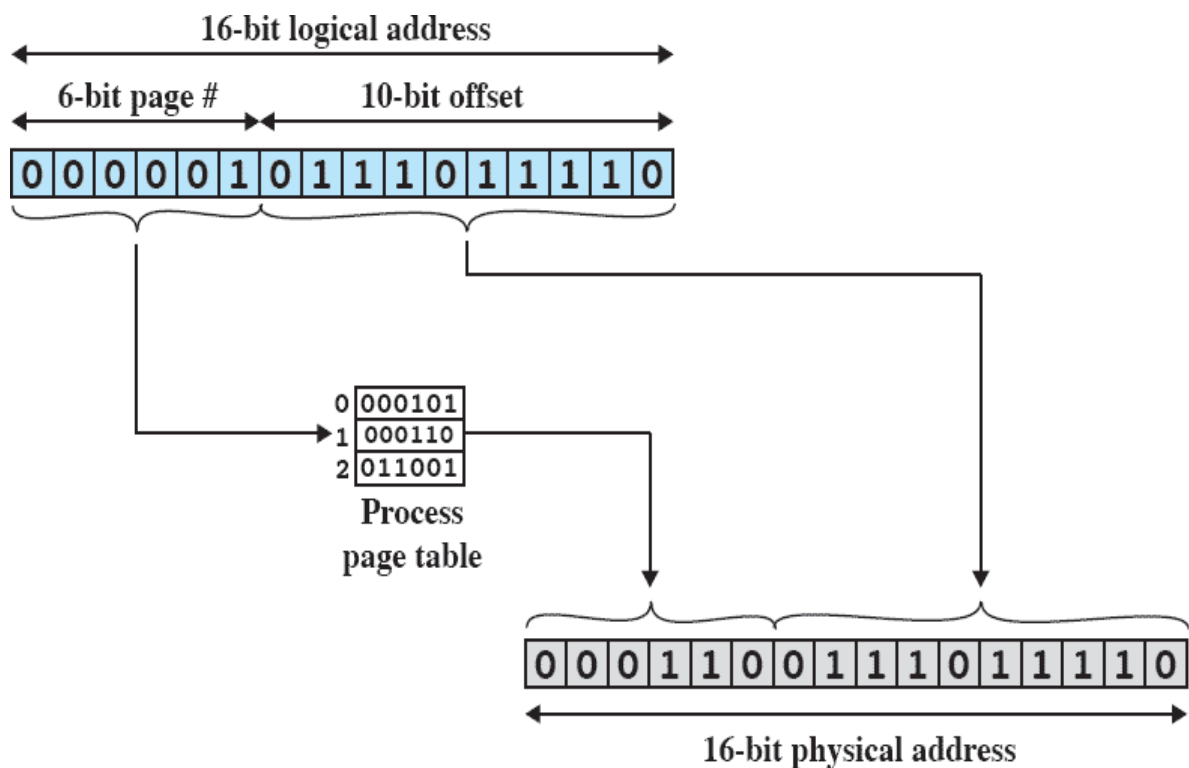
- The operating system maintains a copy of the page table for each process. Paging therefore increases the context-switch time.

Example:

In this example, we have the logical address 0000010111011110, which is page number 1, offset 478.

Suppose that this page is residing in main memory frame 6 = binary 000110.

Then the physical address is frame number 6, offset 478 = 0001100111011110



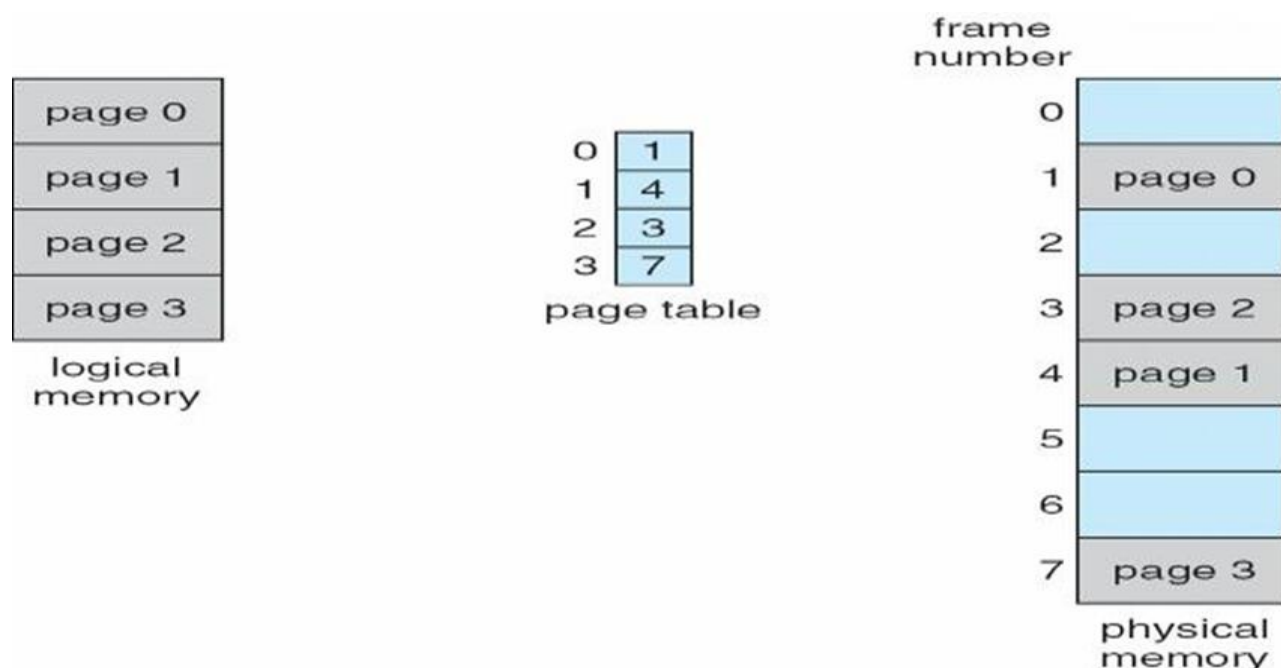
(a) Paging

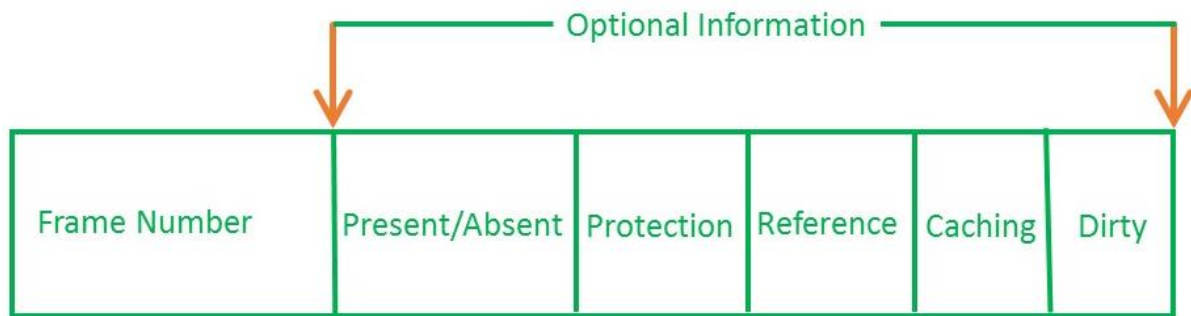
Structure of Page Table:

The data structure that is used by the virtual memory system in the operating system of a computer in order to store the mapping between physical and logical addresses is commonly known as **Page Table**.

As we had already told you that the logical address that is generated by the CPU is translated into the physical address with the help of the page table.

Thus page table mainly provides the corresponding frame number (base address of the frame) where that page is stored in the main memory.





PAGE TABLE ENTRY

- **Frame Number** – It gives the **frame number in which the current page you are looking for is present**. The number of bits required depends on the number of frames. Frame bit is also known as address translation bit.

$$\text{Number of bits for frame} = \text{Size of physical memory} / \text{Frame Size}$$

- **Present/Absent Bit:** Present or absent bit says whether a particular page you are looking for is present or absent.

In case it is not present, that is called Page Fault. It is **set to 0** if the corresponding page is not in memory. Sometimes this bit is also known as a **valid/invalid** bit.

- **Protection Bit:** The protection bit says what kind of protection you want on that page. So, these bits are for the protection of the page frame (read, write, etc).
- **Referenced Bit:** Referenced bit will say whether this page has been referred to in the last clock cycle or not. It is set to 1 by hardware when the page is accessed.
- **Caching Enabled/Disabled:** Sometimes we need fresh data.

Let us say the user is typing some information from the keyboard and your program should run according to the input given by the user.

In that case, the information will come into the main memory. Therefore main memory contains the latest information which is typed by the user.

Now if you try to put that page in the cache, that cache will show the old information.

So whenever freshness is required, we don't want to go for caching or many levels of memory.

That is the reason we want to disable caching. So, this bit **enables or disables** caching of the page.

- **Modified Bit:** Modified bit says whether the page has been modified or not.

Modified means sometimes you might try to write something onto the page.

If a page is modified, then whenever you should replace that page with some other page, then the modified information should be kept on the hard disk or it has to be written back or it has to be saved back.

It is set to 1 by hardware on the write-access to a page which is used to avoid writing when swapped out.

Sometimes this modified bit is also called the **Dirty bit**.

Some of the common techniques that are used for structuring the Page table are as follows:

1. Hierarchical Paging
2. Hashed Page Tables
3. Inverted Page Tables

Hierarchical Paging:

- There might be a case where the page table is too big to fit in a contiguous space, so we may have a hierarchy with several levels.
- In this type of Paging the logical address space is broke up into Multiple page tables.
- Hierarchical Paging is one of the simplest techniques and for this purpose, a two-level page table and three-level page table can be used.

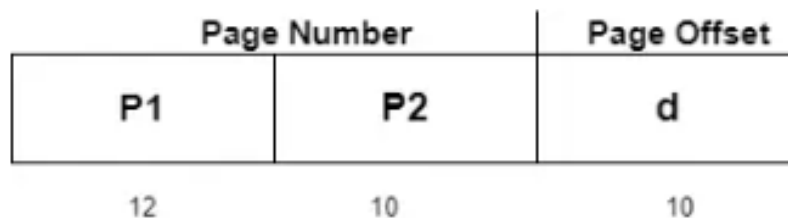
Consider a system having 32-bit logical address space and a page size of 1 KB and it is further divided into:

- Page Number consisting of 22 bits.
- Page Offset consisting of 10 bits.

As we page the Page table, the page number is further divided into :

- Page Number consisting of 12 bits.
- Page Offset consisting of 10 bits.

Thus the Logical address is as follows:



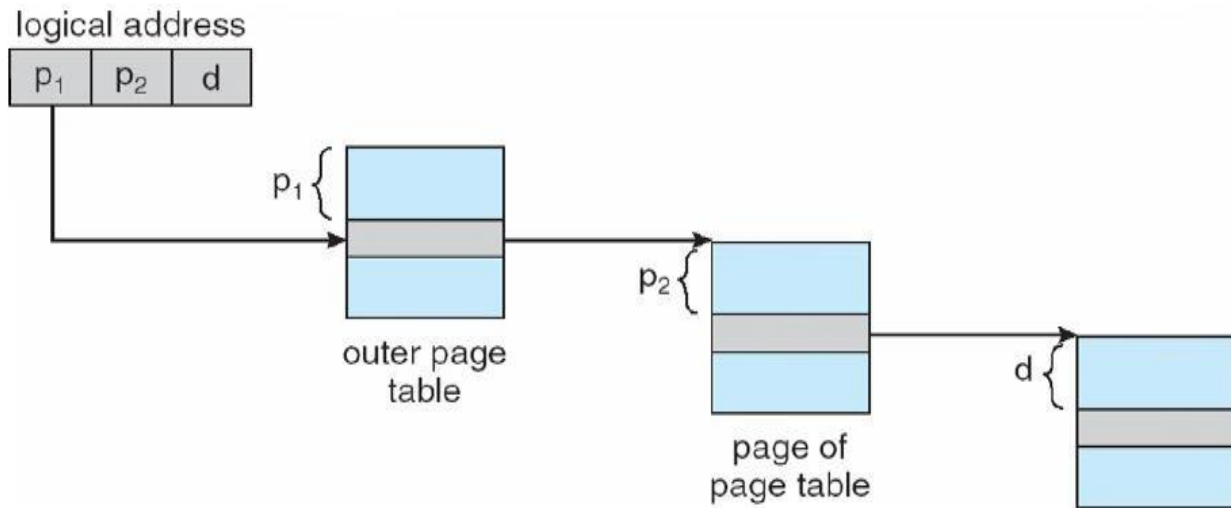
In the above diagram,

P1 is an index into the **Outer Page** table.

P2 indicates the displacement within the page of the **Inner page** Table.

As address translation works from outer page table inward so is known as **forward-mapped Page Table**.

Below given figure below shows the Address Translation scheme for a two-level page table



Three Level Page Table

For a system with 64-bit logical address space, a two-level paging scheme is not appropriate. Let us suppose that the page size, in this case, is 4KB. If in this case, we will use the two-page level scheme then the addresses will look like this:

outer page	inner page	offset
p1	p2	d
42	10	12

Thus in order to avoid such a large table, there is a solution and that is to divide the outer page table, and then it will result in a **Three-level page table**:

2nd outer page	outer page	inner page	offset
p1	p2	p2	d
32	10	10	12

Hashed Page Tables:

This approach is used to handle address spaces that are larger than 32 bits.

- In this virtual page, the number is hashed into a page table.
- This Page table mainly contains a chain of elements hashing to the same elements.

Each element mainly consists of :

1. The virtual page number
2. The value of the mapped page frame.
3. A pointer to the next element in the linked list.

Given below figure shows the address translation scheme of the Hashed Page Table:

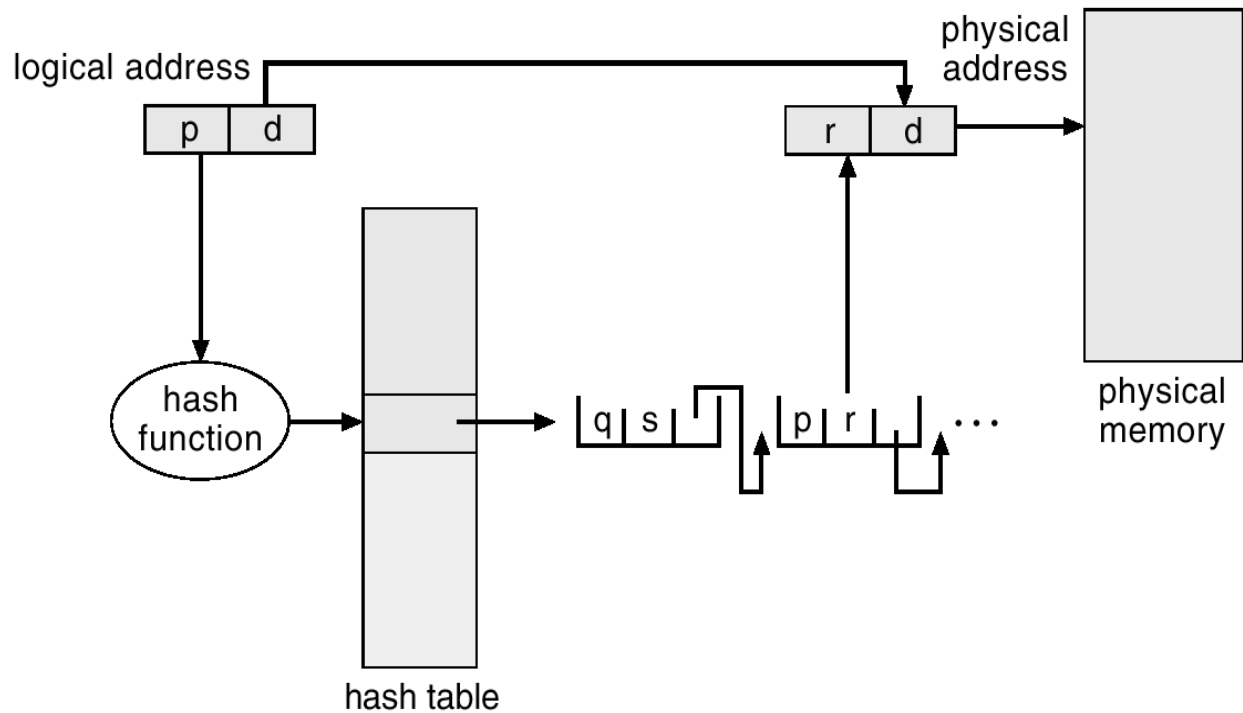


Fig. shows Hashed Page Table

The Virtual Page numbers are compared in this chain searching for a match; if the match is found then the corresponding physical frame is extracted.

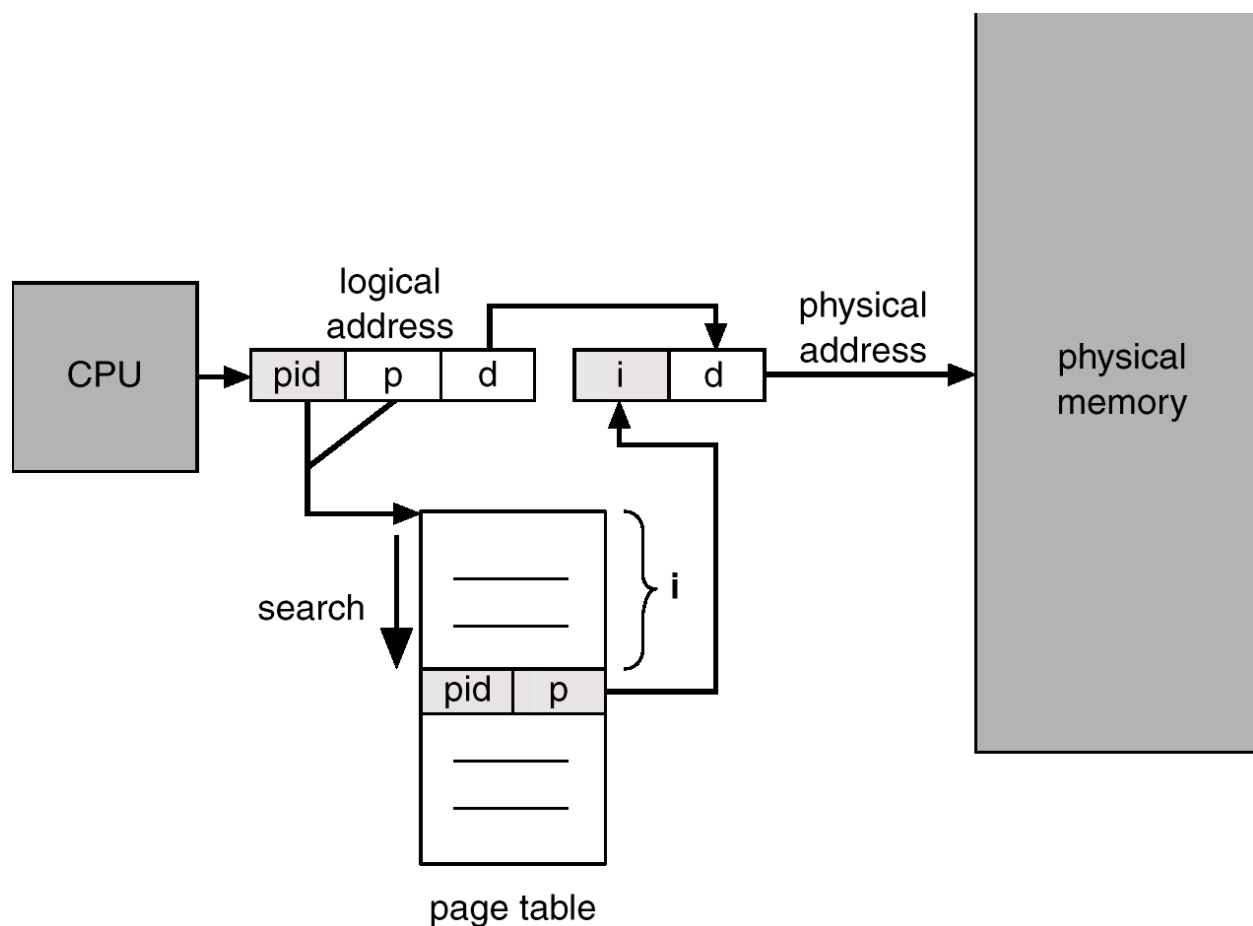
Inverted Page Tables:

The Inverted Page table basically combines A page table and A frame table into a single data structure.

- There is one entry for each virtual page number and a real page of memory

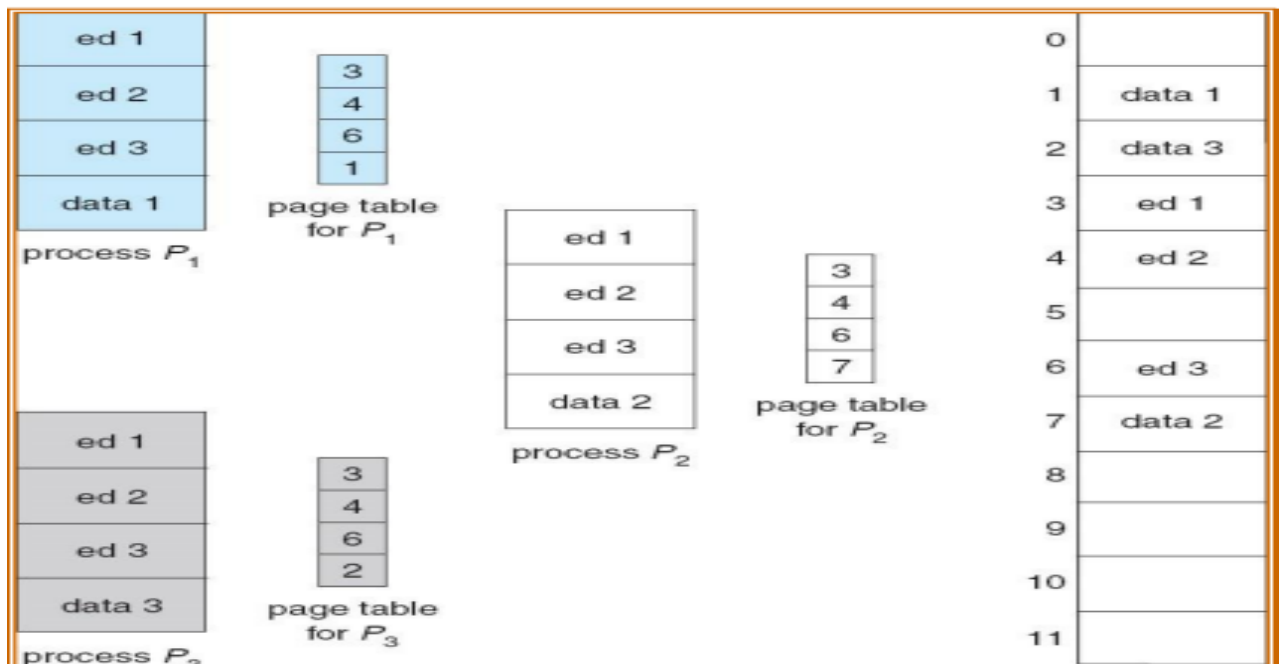
- And the entry mainly consists of the virtual address of the page stored in that real memory location along with the information about the process that owns the page.
- Though this technique decreases the memory that is needed to store each page table; but it also increases the time that is needed to search the table whenever a page reference occurs.

Given below figure shows the address translation scheme of the Inverted Page Table:



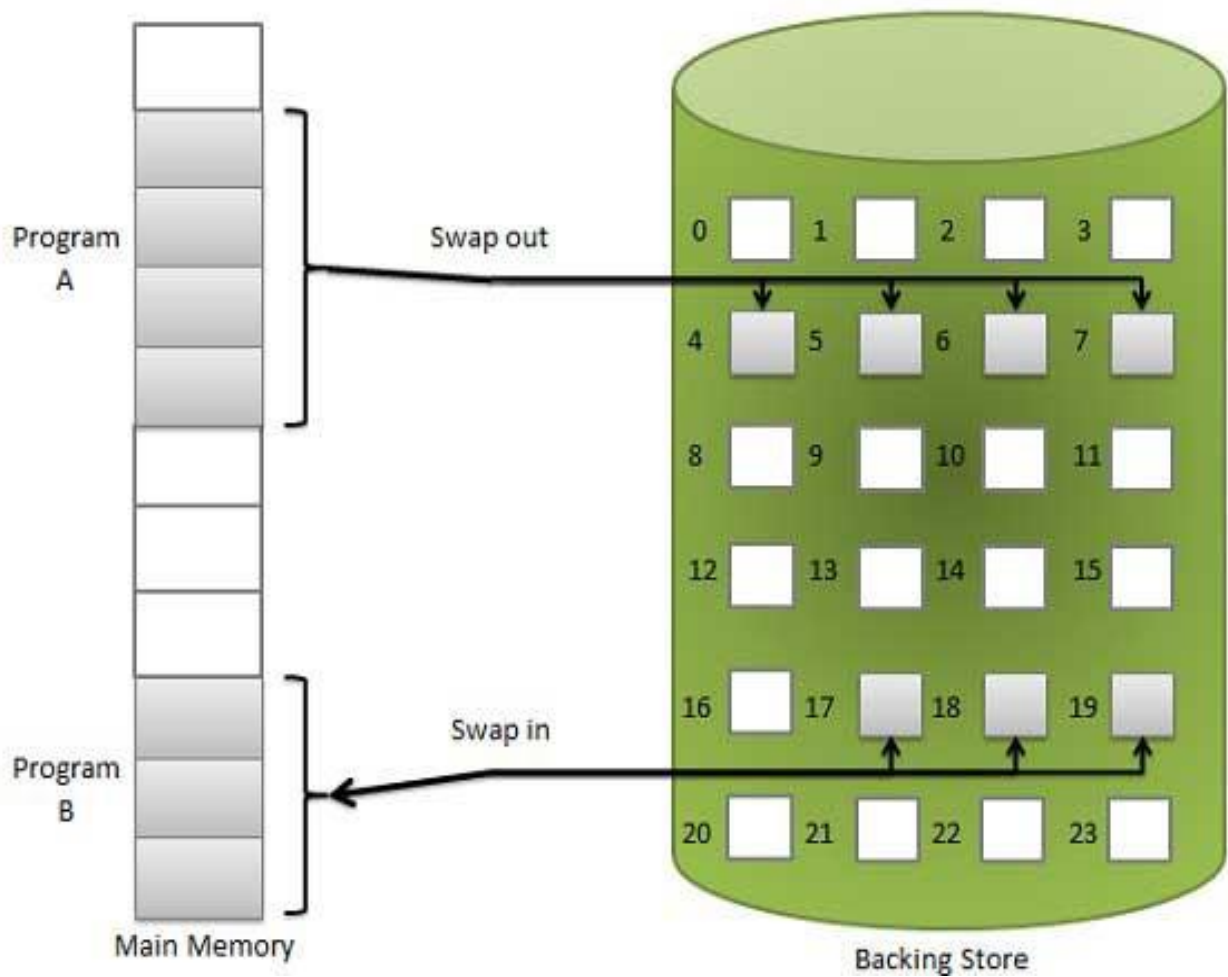
Shared Page Table:

- Shared code
 - o One copy of read-only (reentrant) code shared among processes (i.e., text editors, compilers, window systems).
 - o Shared code must appear in same location in the logical address space of all processes
- Private code and data
 - o Each process keeps a separate copy of the code and data
 - o The pages for the private code and data can appear anywhere in the logical address space.



Demand Paging:

- A demand paging system is quite similar to a paging system with swapping.
- When we want to execute a process, we swap it into memory. Rather than swapping the entire process into memory, however, we use a lazy swapper called pager.
- When a process is to be swapped in, the pager guesses which pages will be used before the process is swapped out again.
- Instead of swapping in a whole process, the pager brings only those necessary pages into memory.
- Thus, it avoids reading into memory pages that will not be used in anyway, decreasing the swap time and the amount of physical memory needed.
- Hardware support is required to distinguish between those pages that are in memory and those pages that are on the disk using the valid-invalid bit scheme.
- Where valid and invalid pages can be checked by checking the bit.
- Marking a page will have no effect if the process never attempts to access the page.
- While the process executes and accesses pages that are memory resident, execution proceeds normally



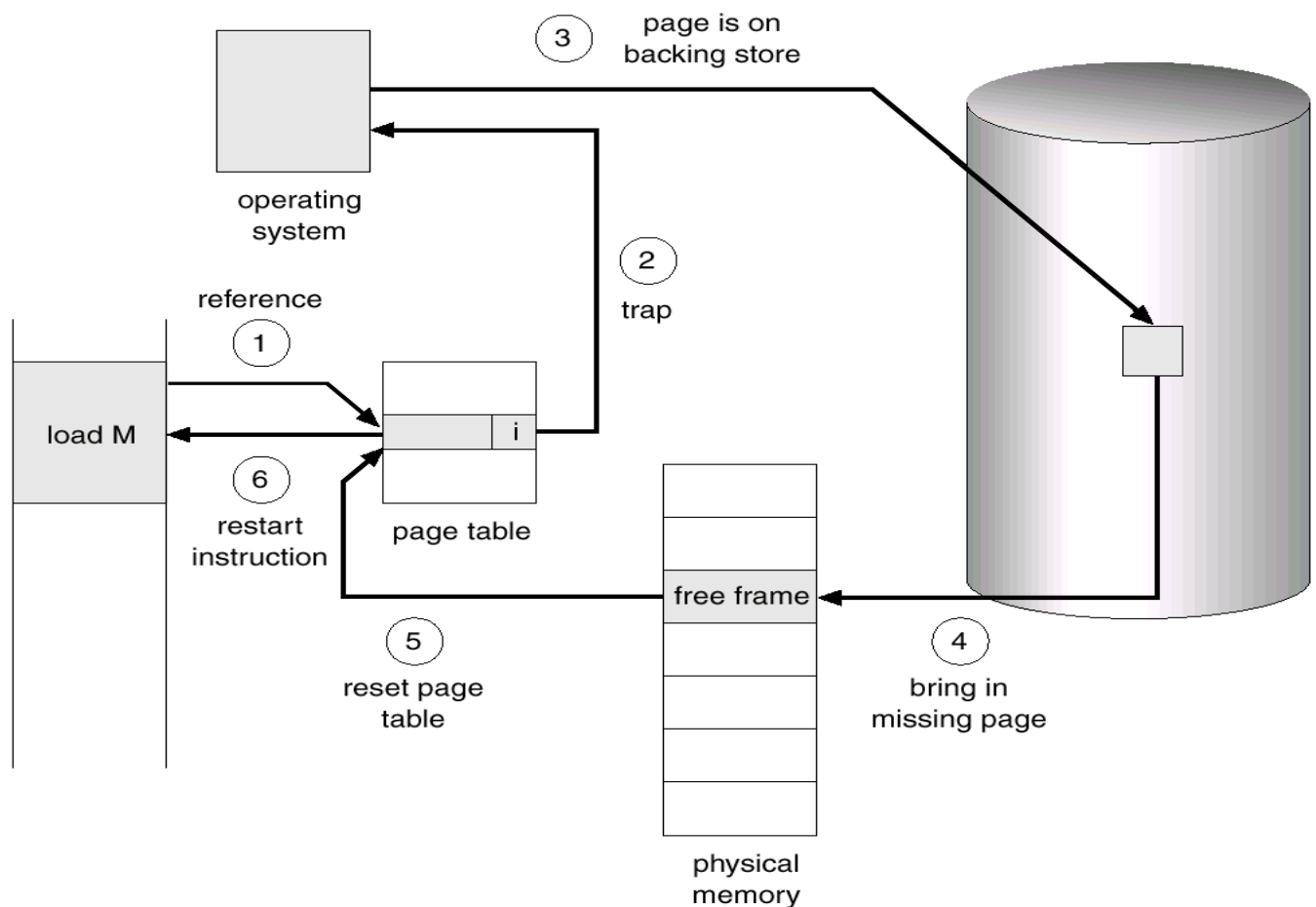
Transfer of a Paged Memory to Contiguous Disk Space

Handling Page Fault:

When the page accessed by the CPU is not found in the main memory, the situation is referred to as Page Fault. If the required page is not loaded into the memory, then a page fault trap arises.

To recover from this, the required page has to be fetched from the secondary memory(hard disk) into the main memory.

So, when page fault occurs then following sequence of events happens:



- The computer hardware traps to the kernel and program counter (PC) is saved on the stack. Current instruction state information is saved in CPU registers.
- An assembly program is started to save the general registers and other volatile information to keep the OS from destroying it.
- Operating system finds that a page fault has occurred and tries to find out which virtual page is needed. Some times hardware register contains this required information. If not, the operating system must retrieve PC, fetch instruction and find out what it was doing when the fault occurred.
- Once virtual address caused page fault is known, system checks to see if address is valid and checks if there is no protection access problem.
- If the virtual address is valid, the system checks to see if a page frame is free. If no frames are free, the page replacement algorithm is run to remove a page.
- If frame selected is dirty, page is scheduled for transfer to disk, context switch takes place, fault process is suspended and another process is made to run until disk transfer is completed.

- As soon as page frame is clean, operating system looks up disk address where needed page is, schedules disk operation to bring it in.
- When disk interrupt indicates page has arrived, page tables are updated to reflect its position, and frame marked as being in normal state.
- Faulting instruction is backed up to state it had when it began and PC is reset. Faulting is scheduled, operating system returns to routine that called it.
- Assembly Routine reloads register and other state information, returns to user space to continue execution.

Translation Lookaside Buffer (TLB):

The Translation Lookaside Buffer (TLB) is a **cache memory used in computer systems to improve the speed** of virtual address translation in the context of paging.

Paging is a memory management scheme used by operating systems to manage memory efficiently by dividing it into fixed-size blocks called pages.

When a program accesses memory, it generates a virtual address, which needs to be translated into a physical address before the data can be accessed in memory.

This translation process involves looking up a page table, which maps virtual addresses to physical addresses.

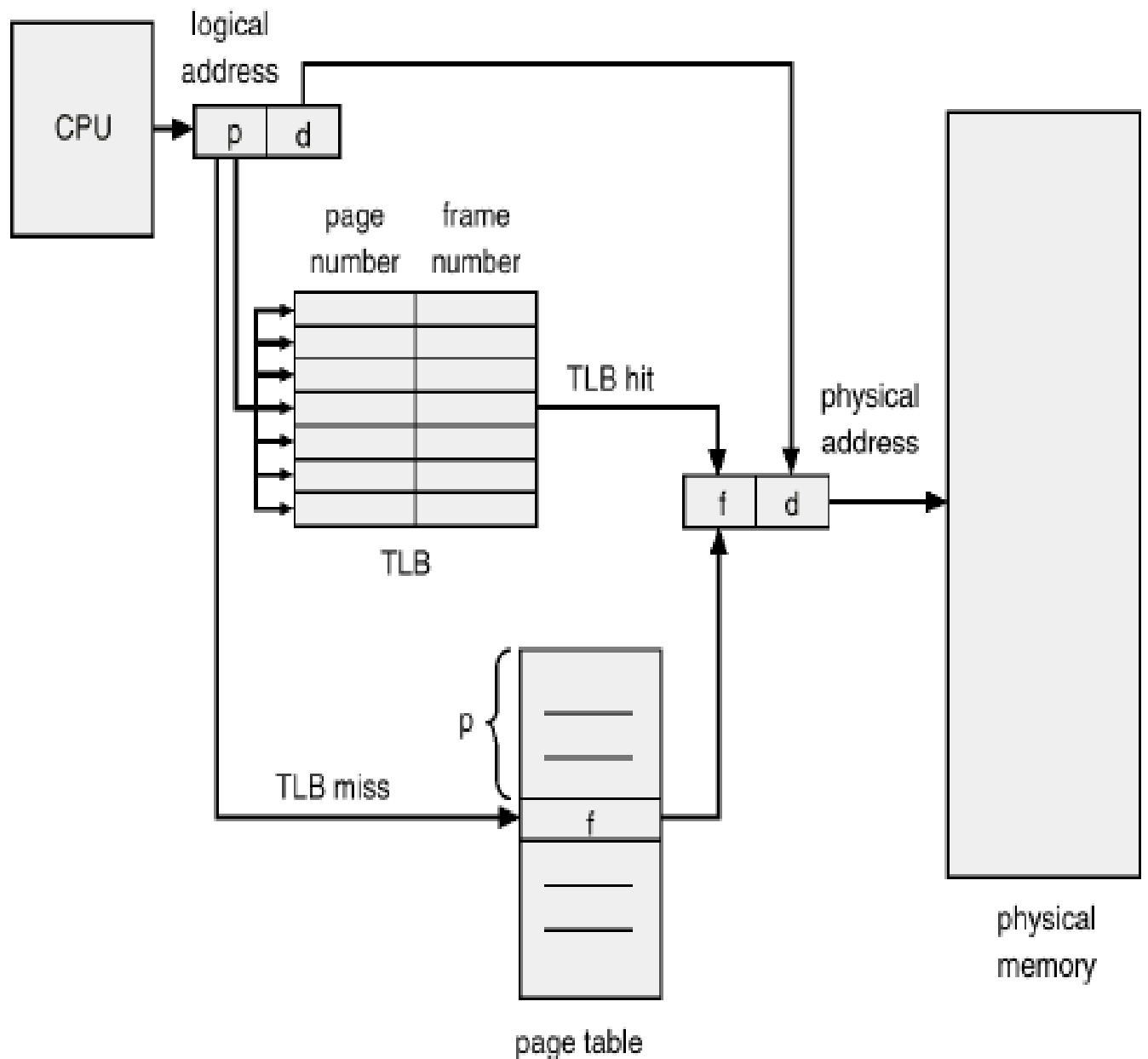
However, this lookup process can be time-consuming, especially when performed for every memory access.

To **speed up this translation process**, the **TLB caches recently accessed virtual-to-physical address mappings**.

When a program accesses memory, the TLB is first checked to see if the virtual address is already mapped to a physical address.

If a mapping is found in the TLB (a TLB hit), the translation can be performed quickly without accessing the page table.

If no mapping is found (a TLB miss), the translation proceeds by accessing the page table, and the resulting mapping is typically stored in the TLB for future use.

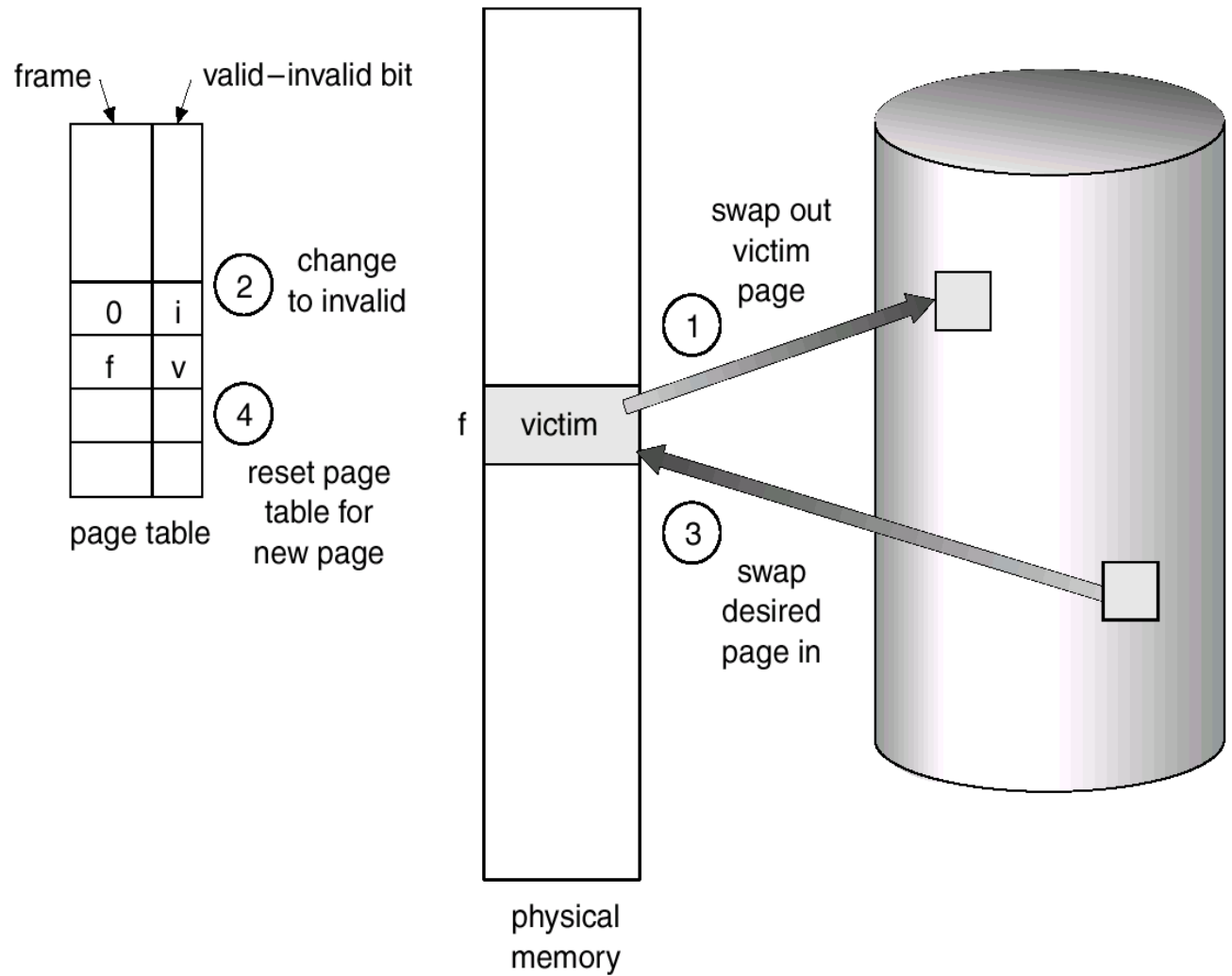


Page Replacement:

- When we over-allocate memory, we need to push out something already in memory. Over- allocation may occur when programs need to fault in more pages than there are physical frames to handle.

Approach:

- If no physical frame is free, find one not currently being touched and free it.
- Steps to follow are:
 - Find the location of the desired page on disk.
 - Find a free frame:
 - If there is a free frame, use it.
 - If there is no free frame, use a page replacement algorithm to select a victim frame.
 - Read the desired page into the (newly) free frame. Update the page and frame tables.
 - Restart the process.



Page Replacement Algorithm:

There are many different page replacement algorithms. We evaluate an algorithm by running it on a particular string of memory reference and computing the number of page faults. The string of memory references is called reference string.

Some of the page replacement algorithms are

1. FIFO page replacement
2. Optimal page replacement
3. LRU page replacement
4. Most Recently Used (MRU)
5. Least Frequently Used (LFU)
6. Second chance page replacement algorithm

First In First out Page replacement algorithm:

- The oldest and simplest page in the physical memory is the one selected for replacement.
- Very simple to implement.
- Keep a list on a page fault, the page at the head is removed and the new page added to the tail of the list.

- This algorithm associates with each page the time that page was brought into memory
- When a page must be replaced, the oldest page is chosen

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2	2	2	4	4	4	0	0	0	7	7	7
	0	0	0	3	3	3	2	2	2	1	1	1	0	0
		1	1	1	0	0	0	3	3	3	2	2	2	1

page frames

Total numbers of page fault = 15

Fault rate = number of page fault/numbers of reference string

$$= 15/20 = \frac{3}{4} = 0.75$$

To determine the number of page faults for a particular reference string and page replacement algorithm, we also need to know the number of page frames available. As the number of frames available increase, the number of page faults will decrease.

FIFO Anomaly:

Belady's(or FIFO) Anomoly—Certain page reference patterns actually cause **more page faults** when number of page frames allocated to a process is **increased**

Page reference	Result	FIFO page replacement with three pages available				FIFO page replacement with four pages available			
A	Fault	A	–	–	Fault	A	–	–	–
B	Fault	B	A	–	Fault	B	A	–	–
C	Fault	C	B	A	Fault	C	B	A	–
D	Fault	D	C	B	Fault	D	C	B	A
A	Fault	A	D	C	No fault	D	C	B	A
B	Fault	B	A	D	No fault	D	C	B	A
E	Fault	E	B	A	Fault	E	D	C	B
A	No fault	E	B	A	Fault	A	E	D	C
B	No fault	E	B	A	Fault	B	A	E	D
C	Fault	C	E	B	Fault	C	B	A	E
D	Fault	D	C	E	Fault	D	C	B	A
E	No fault	D	C	E	Fault	E	D	C	B
Three "no faults"					Two "no faults"				

All pages frames initially empty

	0	1	2	3	0	1	4	0	1	2	3	4	
Youngest page		0	1	2	3	0	1	4	4	4	2	3	3
			0	1	2	3	0	1	1	1	4	2	2
Oldest page				0	1	2	3	0	0	0	1	4	4
	P	P	P	P	P	P	P			P	P		

9 Page faults

(a)

	0	1	2	3	0	1	4	0	1	2	3	4	
Youngest page		0	1	2	3	3	3	4	0	1	2	3	4
			0	1	2	2	2	3	4	0	1	2	3
Oldest page				0	1	1	1	2	3	4	0	1	2
				0	0	0	1	2	3	4	0	1	
	P	P	P	P			P	P	P	P	P	P	

10 Page faults

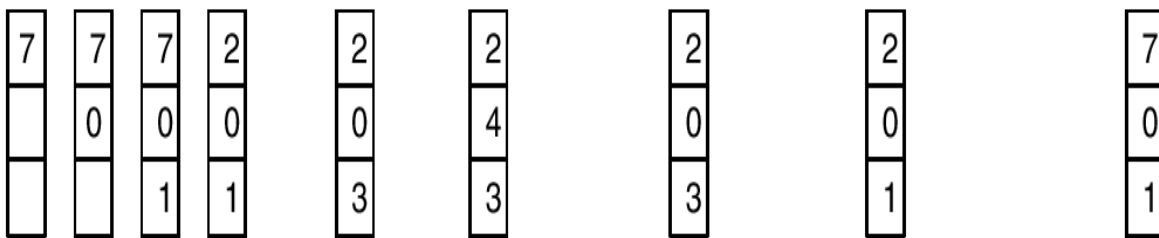
(b)

Optimal Page Replacement:

- Replace page that will not be used for longest period of time.
- Use of this page replacement algorithm guarantees the lowest possible page rate for a fixed number of frames.

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames

Total numbers of page fault = 9

Fault rate = number of page fault/numbers of reference string

$$= 9/20 = 0.45$$

Least Recently Used (LRU) Algorithm:

- LRU page replacement—Exploits temporal locality by replacing the page that has spent the longest time in memory without being referenced
- Can provide better performance than FIFO
- Increased system overhead
- LRU can perform poorly if the least-recently used page is the next page to be referenced by a program that is iterating inside a loop that references several pages

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2	2	4	4	4	0	1	1	1
	0	0	0	0	0	0	3	3	3	0	0
		1	1	3	3	2	2	2	2	2	7

page frames

Total numbers of page fault = 12

Fault rate = number of page fault/numbers of reference string

$$= 12/20 = 0.6$$

Most Recently Used (MRU):

In this algorithm, page will be replaced which has been used recently. Belady's anomaly can occur in this algorithm.

Page reference	7,0,1,2,0,3,0,4,2,3,0,3,2,3														No. of Page frame - 4
7	0	1	2	0	3	0	4	2	3	0	3	2	3		
			2	2	2	2	2	2	3	0	3	2	3		
		1	1	1	1	1	1	1	1	1	1	1	1		
	0	0	0	0	3	0	4	4	4	4	4	4	4		
7	7	7	7	7	7	7	7	7	7	7	7	7	7		
Miss	Miss	Miss	Miss	Hit	Miss	Miss	Miss	Hit	Miss	Miss	Miss	Miss	Miss		
Total Page Fault = 12															

Least Frequently Used (LFU):

The LFU page replacement algorithm stands for the **Least Frequently Used**.

In the LFU page replacement algorithm, the page with the least visits in a given period of time is removed.

It replaces the least frequently used pages. If the frequency of pages remains constant, the page that comes first is replaced first.

Example:

Reference String:

7 0 2 4 3 1 4 7 2 0 4 3 0 3 2 7

String	7	0	2	4	3	1	4	7	2	0	4	3	0	3	2	7
Frame 3			2	2	2	1	1	1	2	2	2	3	3	3	3	3
Frame 2		0	0	0	3	3	3	7	7	0	0	0	0	0	2	7
Frame 1	7	7	7	4	4	4	4	4	4	4	4	4	4	4	4	4
Miss/Hit	M	M	M	M	M	M	H	M	M	M	H	M	H	H	M	M

Second-Chance Page Replacement Algorithm:

The basic algorithm of second-chance replacement is a FIFO replacement algorithm. When a page has been selected, however, we inspect its reference bit. If the value is 0, we proceed to replace this page. If the reference bit is set to 1, however, we give that page a second chance and move on to select the next FIFO page. When a page gets a second chance, its reference bit is cleared and its arrival time is reset to the current time. Thus, a page that is given a second chance will not be replaced until all other pages are replaced (or given second chances). In addition, if a page is used often enough to keep its reference bit set, it will never be replaced.

Working Set Page Replacement Algorithm:

The set of pages that a process is currently using is known as its working set.

If the entire working set is in memory, the process will run without causing many faults until it moves into another execution phase.

If the the available memory is too small to hold the entire working set, the process will cause many page faults and run slowly.

To implement the working set model, it is necessary for the operating system to keep track of which pages are in working set.

Having this information also immediately leads to a possible page replacement algorithm: when a page fault occurs, find a page not in the working set and evict it.

Concept of Locality of Reference:

Locality of Reference refers to a phenomenon in which a computer program tends to access same set of memory locations for a particular time period.

In other words, **Locality of Reference** refers to the tendency of the computer program to access instructions whose addresses are near one another.

1. **Spatial Locality:** This aspect of locality of reference suggests that if a program accesses a particular memory location, it is likely to access nearby memory locations in the near future.
2. **Temporal Locality:** Temporal locality refers to the tendency of programs to access the same memory locations repeatedly over a short period of time.

When a program accesses a memory location, it is likely to access the same location again in the near future.

Thrashing:

- In case, if the page fault and swapping happens very frequently at a higher rate, then the operating system has to spend more time swapping these pages. This state in the operating system is termed thrashing. Because of thrashing the CPU utilization is going to be reduced.

- Let's understand by an example, if any process does not have the number of frames that it needs to support pages in active use then it will quickly page fault.
- And at this point, the process must replace some pages. As all the pages of the process are actively in use, it must replace a page that will be needed again right away.
- Consequently, the process will quickly fault again, and again, and again, replacing pages that it must bring back in immediately. This high paging activity by a process is called thrashing.
- During thrashing, the CPU spends less time on some actual productive work spend more time swapping.

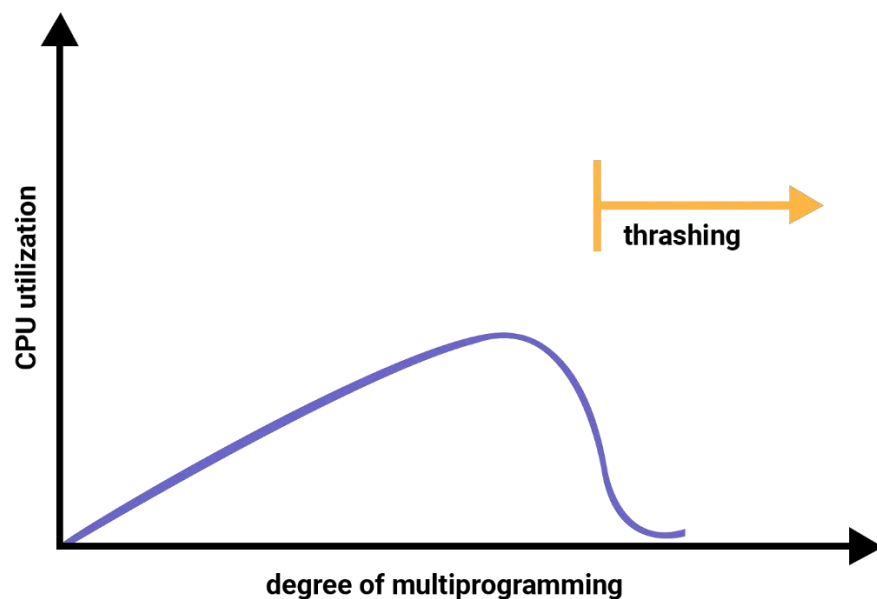


Fig. Thrashing

Segmentation:

Segmentation is another way of dividing the addressable memory. It is another scheme of memory management and it generally supports the user view of memory. The Logical address space is basically the collection of segments. Each segment has a name and a length.

It is not required that all segments of all programs be of the same length, although there is a maximum segment length. As with paging, a logical address using segmentation consists of two parts, in this case a *segment number* and an *offset*.

Because of the use of unequal-size segments, segmentation is similar to dynamic partitioning. In segmentation, a program may occupy more than one partition, and these partitions need not be contiguous. Segmentation eliminates internal fragmentation but, like dynamic partitioning, it suffers from external fragmentation. However, because a process is broken up into a number of smaller pieces, the external fragmentation should be less.

Logical address consists of a two tuple: <segment-number, offset>,

- *Segment table* –maps two-dimensional physical addresses; each table entry has:

♦base –contains the starting physical address where the segments reside in memory.

♦limit –specifies the length of the segment.

- *Segment-table base register (STBR) points to the segment table's location in memory.*
- *Segment-table length register (STLR) indicates number of segments used by a program; segment number s is legal if $s < STLR$.*

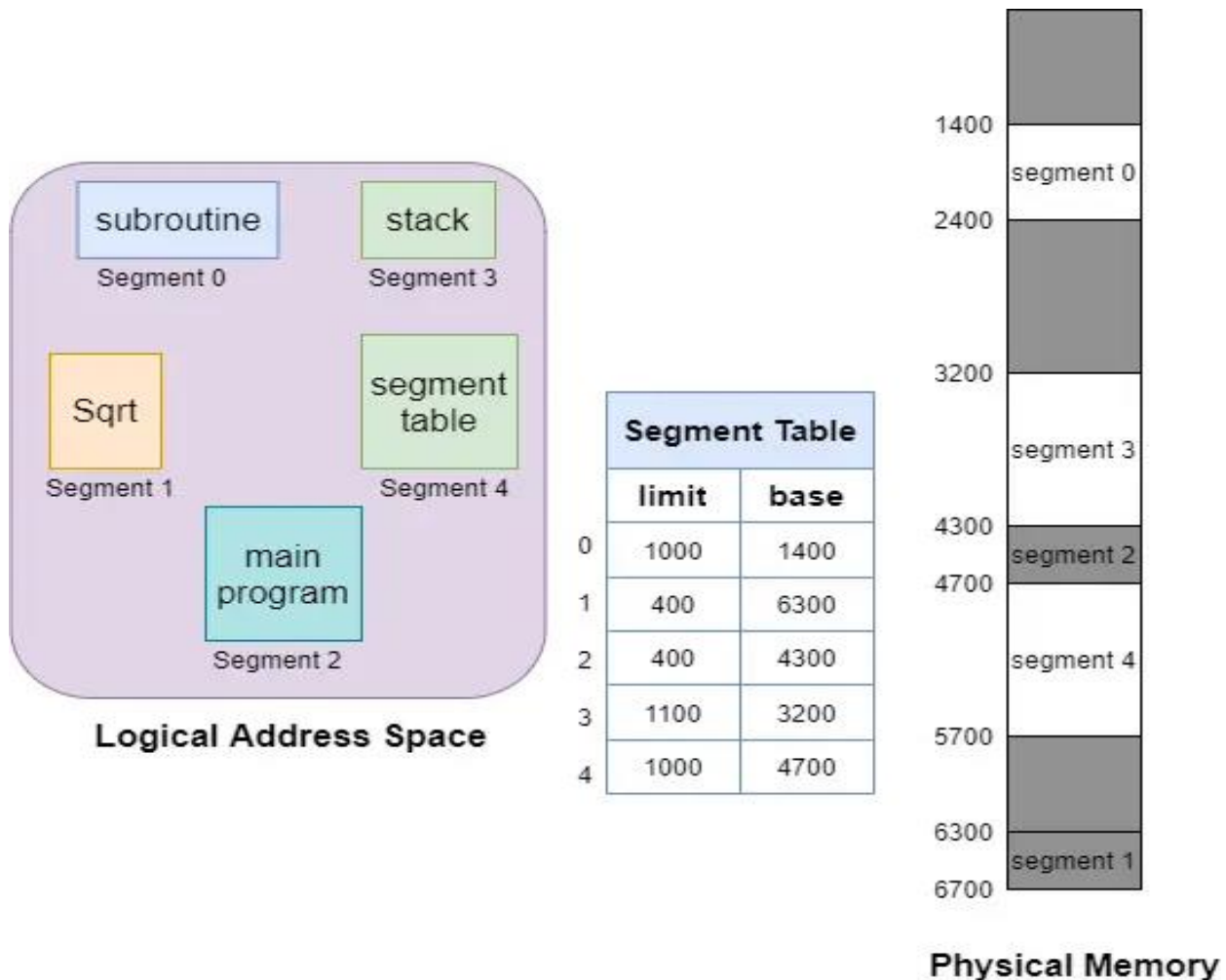


Fig. Example of Segmentation

S.N	Paging	Segmentation
1	In Paging, we break a process address space into blocks known as pages.	In the case of Segmentation, we break a process address space into blocks known as sections.
2	The pages are blocks of fixed size.	The sections are blocks of varying sizes.
3	This technique is comparatively much faster in accessing memory.	This technique is comparatively much slower in accessing memory than Paging.
4	The available memory determines the individual page sizes.	The user determines the individual segment sizes.
5	A logical address divides into page offset and page number in the case of Paging.	A logical address divides into section offset and section number in the case of Segmentation.

6	In the case of Paging, the page table leads to the storage of the page data.	In the case of Segmentation, the segmentation table leads to the storage of the segmentation data.
---	--	--

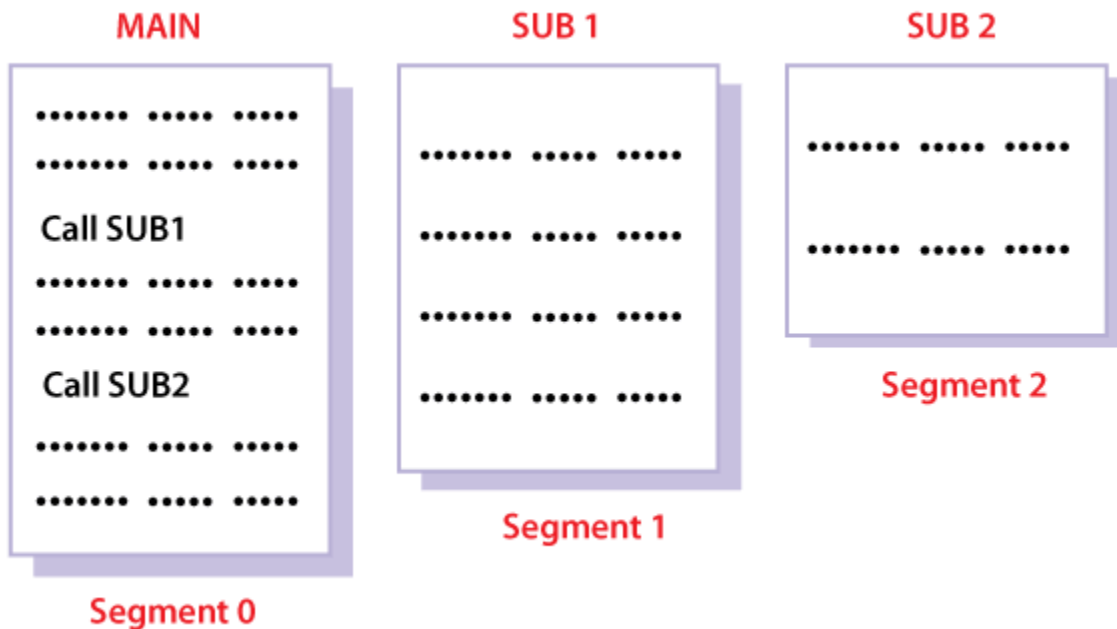
Need of Segmentation:

Till now, we were using Paging as our main memory management technique. **Paging is more close to the Operating system rather than the User.** It divides all the processes into the form of pages regardless of the fact that a process can have some relative parts of functions which need to be loaded in the same page.

Operating system doesn't care about the User's view of the process. It may divide the same function into different pages and those pages may or may not be loaded at the same time into the memory. It decreases the efficiency of the system.

It is better to have segmentation which divides the process into the segments.

Each segment contains the same type of functions such as the main function can be included in one segment and the library functions can be included in the other segment.



Advantages of Segmentation

- No internal fragmentation
- Average Segment Size is larger than the actual page size.
- Less overhead
- It is easier to relocate segments than entire address space.
- The segment table is of lesser size as compared to the page table in paging.

Disadvantages

- Maintaining a segment table for each process leads to overhead
- This technique is expensive.
- The time is taken in order to fetch the instruction increases since now two memory accesses are required.
- Segments are of unequal size in segmentation and thus are not suitable for swapping.
- This technique leads to external fragmentation as the free space gets broken down into smaller pieces along with the processes being loaded and removed from the main memory then this will result in a lot of memory waste.

Segmentation with Paging:

MULTICS (Multiplexed Information and Computing Service) was an influential operating system developed in the 1960s and 1970s, which introduced several innovative concepts in operating system design, including segmentation with paging.

In MULTICS, segmentation with paging was used as a memory management technique to provide a flexible and efficient way of organizing and accessing memory. Here's how segmentation with paging worked in MULTICS:

1. Segmentation:

- Memory was divided into variable-sized segments, each representing a logical unit of the program, such as code, data, or stack.
- Segments were defined by segment descriptors, which contained information about the segment's size, location in physical memory, access permissions, and other attributes.
- Segmentation provided a way to logically divide the address space of a process into manageable units, allowing for easier program organization and memory protection.

2. Paging:

- Within each segment, memory was further divided into fixed-sized pages. Each page typically contained 1024 words (or 4096 bytes) in MULTICS.

- Paging allowed for efficient memory allocation and management by breaking down memory into smaller, uniform-sized units.
- Pages were managed by the paging system, which kept track of page tables containing the mapping between logical (virtual) addresses and physical addresses.

3. Segmentation with Paging:

- Segmentation and paging were combined to provide a flexible and efficient memory management scheme.
- Each segment had its own page table, which mapped the segment's logical addresses to physical addresses of pages in memory.
- This allowed each segment to be independently managed, with its own protection and sharing attributes.
- When a program accessed a memory address, the segmentation system determined the segment to which the address belonged. Then, the paging system within that segment's page table translated the logical address to a physical address, allowing the program to access the corresponding memory location.

4. Benefits:

- Segmentation with paging provided flexibility, allowing programs to be divided into logical units (segments) while also enabling efficient memory allocation and protection.
- It supported demand paging, where pages were brought into memory only when needed, reducing the memory footprint of programs and improving overall system performance.
- Segmentation with paging also facilitated memory protection and sharing by providing mechanisms to control access to segments and pages.